

**Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»**

Кафедра

«Информационные системы и технологии»

(наименование кафедры)

У Т В Е Р Ж Д А Ю

Заведующий кафедрой

«Информационные системы и технологии»

(наименование кафедры)

(ученая степень, ученое звание)

(подпись)

(И.О. Фамилия)

« » 20 г.

**О Т Ч Е Т
О ПРОХОЖДЕНИИ ПРАКТИКИ**

производственная

(вид практики: учебная / производственная)

Учебная практика: Научно-исследовательская работа (получение первичных
навыков научно-исследовательской работы)

(тип практики)

Обучающийся

Прикладная информатика в экономике

(название направления подготовки / специальности)

3

(курс)

31ПРД4310

(группа)

« » 2026г.

(дата)

(подпись)

Д. К. Иванюк

(И.О. Фамилия)

Руководитель практики от МФЮА

(должность)

(наименование кафедры)

(ученая степень, ученое звание)

(оценка прописью)

« » 20 г.
(дата)

(подпись)

(И.О. Фамилия)

Руководитель практики от профильной организации

(должность)

(название профильной организации)

(ученая степень, звание)

« » 2026г.
(дата)

(подпись)

М.П.

(И.О. Фамилия)

ЯРОСЛАВЛЬ 2026

СОДЕРЖАНИЕ

Введение	3
1. Характеристика Ярославского гарнизонного военного суда ЯГВС	5
1.1. Общие сведения об организации	5
1.2. Организационная структура управления	8
1.3. Информационная инфраструктура предприятия	10
1.4. Основные бизнес-процессы предприятия	11
2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	12
2.1. Особенности предметной области	12
2.2. Моделирование процесса AS-IS	15
2.3. Выявленные проблемы и ограничения	17
3. ОБОСНОВАНИЕ РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ	18
3.1. Цели автоматизации	18
3.2. Анализ альтернативных решений	19
3.3. Техничко-экономическое обоснование	20
4. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ СИСТЕМЫ	21
4.1. Назначение и основания разработки	21
4.2. Функциональные требования	22
4.3. Нефункциональные требования	23
4.4. Требования к данным, техническим средствам и безопасности	26
5. ПРОЕКТИРОВАНИЕ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ	27
5.1. Модель процесса ТО-ВЕ	27
5.2. Роли пользователей и варианты использования	28
5.3. Архитектура системы и развертывание	30
5.4. Проектирование базы данных	31
5.5. Проектирование классов и алгоритмов	35
5.6. Жизненный цикл программы	37
6. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ЯДРА	39
6.1. Выбор инструментальных средств	39
6.2. Реализация структуры базы данных SQL Server	40
6.3. Реализация хранимых процедур	41
6.5. Связь требований, алгоритмов и реализации	43
7. ТЕСТИРОВАНИЕ И ОТЛАДКА	44
7.1. Методы тестирования	44
7.2. Тест-кейсы	45
7.3. Результаты тестирования	47
8. ОЦЕНКА КАЧЕСТВА ПРОГРАММЫ	48
8.1. Характеристики качества по ГОСТ ИСО/МЭК 25010-2026	48
ЗАКЛЮЧЕНИЕ	51
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53

Введение

Производственная технологическая практика является важным этапом подготовки обучающегося по направлению 09.03.03 «Прикладная информатика». Ее содержание связано с закреплением навыков анализа предметной области, формирования требований к информационной системе, проектирования базы данных, подготовки технической документации и реализации программного ядра прикладного решения.

Базой прохождения практики является Ярославский гарнизонный военный суд — государственное учреждение, осуществляющее правосудие в отношении военнослужащих и лиц, проходящих военные сборы. Для суда критически важны актуальные сведения об учёте рабочего времени сотрудников аппарата суда (секретарей, помощников судей, специалистов, работников отдела обеспечения судопроизводства), так как ошибки в этих данных приводят к нарушению трудового законодательства, неправильному начислению заработной платы и снижению качества управленческих решений.

Актуальность работы определяется необходимостью перехода от разрозненного учёта в бумажных журналах и ручного заполнения табелей к единой автоматизированной системе учёта рабочего времени. Разрабатываемая система должна обеспечивать фиксацию времени прихода и ухода сотрудников, ведение единой базы данных сотрудников и должностей, разграничение прав доступа (администратор / сотрудник) и формирование отчётов за произвольный период.

Ожидаемые выгоды от внедрения системы:

- Рост ответственности сотрудников — автоматическая фиксация времени прихода и ухода исключает возможность субъективной оценки, каждый сотрудник понимает, что его фактическое присутствие на рабочем месте объективно фиксируется системой.
- Сокращение времени «простоя» — наличие объективного учёта дисциплинирует сотрудников, снижая количество опозданий, преждевременных уходов и нерегламентированных перерывов.
- Упрощение механизма штрафов и премий — автоматически сформированные отчёты о фактически отработанном времени служат объективной основой для начисления стимулирующих выплат добросовестным сотрудникам и применения дисциплинарных мер к нарушителям трудовой дисциплины.

- Снижение временных затрат на формирование отчётности — формирование табеля учёта рабочего времени за период сокращается с 2 часов до 3 секунд.
- Исключение ошибок ручного ввода — автоматический сбор и обработка данных исключают человеческий фактор при учёте рабочего времени.

Цель практики — разработать проект автоматизированной системы учёта рабочего времени сотрудников Ярославского гарнизонного военного суда, оформить проектные решения в соответствии с требованиями стандартов и реализовать программное ядро на языке C++/CLI с использованием базы данных Microsoft SQL Server.

Объектом исследования являются бизнес-процессы Ярославского гарнизонного военного суда, связанные с учётом рабочего времени сотрудников. Предметом исследования является автоматизация процессов учёта рабочего времени с применением реляционной базы данных и прикладного программного обеспечения.

1. Характеристика Ярославского гарнизонного военного суда

ЯГВС

1.1. Общие сведения об организации

Полное наименование: Ярославский гарнизонный военный суд

Сокращённое наименование: ЯГВС

Правовой статус: Федеральный суд общей юрисдикции, входит в судебную систему Российской Федерации, относится к системе военных судов.

Местонахождение: 150999, Ярославская область, г. Ярославль, ул. Карла Либкнехта, д. 20/9

Контактная информация:

- Телефон: 8 (4852) 40-23-53
- Адрес электронной почты: yargvs.jrs@sudrf.ru
- Официальный сайт: <http://yargvs.jrs.sudrf.ru>

Территориальная юрисдикция: Юрисдикция Ярославского гарнизонного военного суда распространяется на территории Ярославской области и Костромской области.

Историческая справка: Военный трибунал Ярославского гарнизона был сформирован в годы Великой Отечественной войны. Днём основания суда считается 5 декабря 1942 года. Первоначально суд состоял из двух человек – председателя трибунала и вольнонаёмного секретаря.

Штатная численность: Штатная численность включает председателя суда, заместителя председателя, судей, помощников судей, секретарей судебных заседаний, специалистов, работников отдела обеспечения судопроизводства, архивариусов, консультантов, администратора суда, системных администраторов и технический персонал.

Ярославский гарнизонный военный суд рассматривается как объект автоматизации. В организации имеется локальная вычислительная сеть, объединяющая рабочие компьютеры сотрудников. Для целей учебной практики предполагается использование двух компьютеров: один выполняет роль сервера базы данных (Microsoft SQL Server), второй — роль клиентского рабочего места с установленным приложением. Такая

конфигурация позволяет предложить клиент-серверную архитектуру без необходимости закупки дополнительного выделенного серверного оборудования.

Основными объектами учёта в системе являются:

- сотрудники суда (работники аппарата, технический персонал);
- должности и организационная структура;
- время прихода и ухода сотрудников;
- отчётные периоды и формирование табелей рабочего времени.

Ключевые особенности организации, влияющие на проектирование системы:

1. Ярославский гарнизонный военный суд входит в систему федеральных судов общей юрисдикции и является судом первой инстанции по делам, подсудным военным судам.

2. Суд рассматривает дела, связанные с деятельностью воинских частей, учреждений и формирований, дислоцирующихся на территории Ярославской и Костромской областей.

3. Аппарат суда обеспечивает деятельность судей и осуществляет организационно-техническое обеспечение судопроизводства.

4. Суд подчиняется вышестоящим судебным инстанциям: апелляционному военному суду и кассационному военному суду.

5. Финансирование и материально-техническое обеспечение осуществляется за счёт средств федерального бюджета через Судебный департамент при Верховном Суде Российской Федерации.

6. Для суда критически важны актуальные сведения об учёте рабочего времени сотрудников аппарата суда, так как ошибки в этих данных приводят к нарушению трудового законодательства, неправильному начислению заработной платы и снижению качества управленческих решений.

Таблица 1. Общая характеристика Ярославского гарнизонного военного суда

Показатель	Характеристика
Наименование организации	Ярославский гарнизонный военный суд (ЯГВС)

Продолжение таблицы 1. Общая характеристика Ярославского гарнизонного военного суда

Показатель	Характеристика
Сфера деятельности	Отправление правосудия в отношении военнослужащих и лиц, проходящих военные сборы
Территориальная юрисдикция	Ярославская область, Костромская область
Основные категории сотрудников	Председатель, заместитель председателя, судьи, начальник отдела, администратор суда, заместитель начальника отдела, помощник судьи, помощник председателя, секретарь судебных заседаний
Информационная база	Локальная сеть, серверный компьютер (SQL Server), клиентские рабочие места
Потребность в автоматизации	Единая система учёта рабочего времени сотрудников, автоматизированное формирование отчётности, контроль трудовой дисциплины

1.2. Организационная структура управления

Организационная структура Ярославского гарнизонного военного суда имеет иерархический функциональный характер. Председатель суда координирует деятельность всех подразделений, принимает управленческие решения на основании отчётов, а также исполняет обязанности судьи — рассматривает дела единолично или коллегиально. Заместитель председателя замещает председателя в его отсутствие и также исполняет обязанности судьи.

Судьи (в настоящее время их роль выполняют председатель и заместитель председателя) осуществляют правосудие, рассматривают дела единолично или коллегиально.

Начальник отдела (отдел обеспечения судопроизводства) руководит работой отдела, организует документооборот и взаимодействие с другими подразделениями, подчиняется непосредственно председателю. Администратор суда обеспечивает организационно-техническое и хозяйственное обеспечение деятельности суда, подчиняется председателю и заместителю председателя.

Заместитель начальника отдела оказывает помощь начальнику отдела в организации работы отдела, замещает его в отсутствие, подчиняется начальнику отдела. Помощник судьи оказывает помощь судьям в подготовке дел к судебному разбирательству, составлении проектов процессуальных документов, подчиняется судьям. Помощник председателя обеспечивает работу председателя, готовит документы и материалы, подчиняется председателю. Секретарь судебных заседаний ведёт протоколы судебных заседаний, извещает участников процесса, подчиняется судьям, начальнику отдела и заместителю начальника отдела.

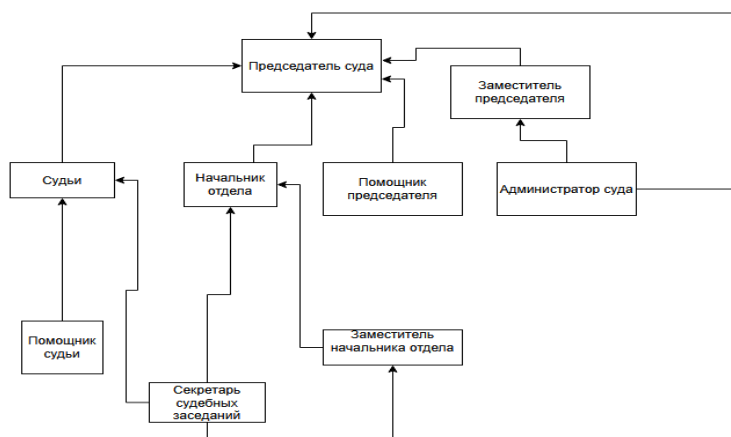


Рисунок 1. Организационная структура Ярославского гарнизонного военного суда.

Таблица 2. Функции участников процессов и подчинение

Роль	Подчиняется	Основные функции
Председатель суда	—	Общее руководство, контроль деятельности, утверждение отчётов, исполнение обязанностей судьи
Заместитель председателя	Председателю	Замещение председателя, исполнение обязанностей судьи
Судья	Председателю, заместителю председателя	Осуществление правосудия, рассмотрение дел
Начальник отдела	Председателю	Руководство отделом обеспечения судопроизводства, контроль явки сотрудников
Заместитель начальника отдела	Начальнику отдела	Организация документооборота, замещение начальника
Администратор суда	Председателю, заместителю председателя	Организационно-техническое обеспечение, настройка пользователей
Помощник судьи	Судьям	Подготовка дел к разбирательству, составление проектов документов
Помощник председателя	Председателю	Обеспечение работы председателя, подготовка документов
Секретарь судебных заседаний	Судьям, начальнику отдела, заместителю начальника отдела	Ведение протоколов, извещение участников процесса

1.3. Информационная инфраструктура предприятия

В Ярославском гарнизонном военном суде имеется локальная вычислительная сеть (ЛВС), объединяющая рабочие компьютеры сотрудников аппарата суда. Для целей автоматизации учёта рабочего времени в рамках учебной практики предлагается использовать следующую конфигурацию информационной инфраструктуры.

Таблица 3. Состав аппаратного обеспечения

Компонент	Назначение	Характеристики (минимальные требования)
Серверный компьютер	Размещение Microsoft SQL Server, хранение базы данных, выполнение хранимых процедур	Процессор: Intel Core i3 / AMD Ryzen 3; ОЗУ: от 4 ГБ; Диск: от 10 ГБ свободного места; ОС: Windows 7/10/11
Клиентский компьютер	Установка клиентского приложения, работа пользователя (администратора) с системой	Процессор: Intel Core i3 / AMD Ryzen 3; ОЗУ: от 4 ГБ; Диск: от 1 ГБ свободного места; ОС: Windows 7/10/11
Локальная вычислительная сеть	Обеспечение взаимодействия между серверным и клиентским компьютером	Сетевые адаптеры, Ethernet-кабель
Резервное хранилище (внешний диск)	Хранение резервных копий базы данных	Объём: от 500 МБ; подключение: USB 2.0/3.0

1.4. Основные бизнес-процессы предприятия

Ярославский гарнизонный военный суд является федеральным органом судебной власти, осуществляющим правосудие на территории Ярославской и Костромской областей. Основные бизнес-процессы (направления деятельности) суда представлены в таблице 4.

Таблица 4. Основные бизнес-процессы Ярославского гарнизонного военного суда

№	Бизнес-процесс	Описание	Результат
1	Рассмотрение уголовных дел	Правосудие по делам о преступлениях, совершённых военнослужащими и лицами, проходящими военные сборы	Судебные акты (приговоры, постановления)
2	Рассмотрение гражданских дел	Разрешение споров, связанных с защитой прав военнослужащих, имущественных и неимущественных отношений	Судебные акты (решения, определения)
3	Рассмотрение административных дел	Дела об административных правонарушениях, отнесённых к подсудности военных судов	Постановления по делам об административных правонарушениях
4	Рассмотрение материалов о грубых дисциплинарных проступках	Дела военнослужащих, за совершение которых может быть назначен дисциплинарный арест	Постановления о назначении дисциплинарного ареста
5	Обеспечение судопроизводства	Организационно-техническое и документационное обеспечение деятельности суда	Сопровождение дел, ведение архива, извещение участников процессов

2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1. Особенности предметной области

Предметная область автоматизации — процесс учёта рабочего времени сотрудников Ярославского гарнизонного военного суда. Данная область характеризуется необходимостью учитывать не только факт присутствия сотрудника на рабочем месте, но и точное время его прихода и ухода. Один и тот же сотрудник в течение рабочего дня может иметь несколько отметок: приход, уход на перерыв, возврат с перерыва, окончание рабочего дня. Однако в рамках текущей версии системы фиксируются только две основные отметки — приход и уход.

Для учёта рабочего времени важны следующие реквизиты:

- Для сотрудника: фамилия, имя, отчество, должность, логин, пароль, роль (администратор / сотрудник), дата приёма на работу, статус активности.
- Для отметок времени: тип отметки (приход / уход), дата и время отметки, идентификатор сотрудника.
- Для отчётности: период (начальная и конечная дата), список сотрудников с их отметками, расчёт отработанного времени.

Для обеспечения подотчётности все критические операции (добавление, редактирование, удаление сотрудников) должны фиксироваться в журнале аудита и требовать подтверждения паролем администратора. Формирование отчётов должно осуществляться в формате CSV для последующей передачи в бухгалтерию и кадровое подразделение.



Рисунок 2. Контекстная диаграмма IDEF0 A-0 процесса учёта рабочего времени

Контекстная диаграмма (IDEF0 A-0) показывает границы автоматизируемого процесса учёта рабочего времени сотрудников Ярославского гарнизонного военного суда:

Входами (слева) являются:

- Данные о сотрудниках — информация о сотрудниках аппарата суда: фамилия, имя, отчество, должность, логин, пароль, роль, дата приёма на работу.
- Отметки времени — фактические отметки о приходе на работу и уходе с работы, фиксируемые в системе.

Управляющими воздействиями (сверху) являются:

- Трудовой кодекс РФ и Правила внутреннего трудового распорядка — нормативно-правовые акты, регламентирующие учёт рабочего времени.
- Техническое задание на разработку — документ, определяющий требования к создаваемой системе.
- Требования безопасности и конфиденциальности — необходимость защиты персональных данных сотрудников и служебной информации.

Механизмами (снизу) являются:

- Администратор суда и начальник отдела — ответственные за управление системой, ведение базы сотрудников и формирование отчётности.
- Сотрудники — пользователи системы, чьё рабочее время учитывается.
- Аппаратное обеспечение — компьютеры, локальная сеть, серверное оборудование.
- Программное обеспечение — разработанное приложение на C++/CLI и СУБД Microsoft SQL Server.

Выходами (справа) являются:

- Журнал отметок времени — база данных всех отметок прихода и ухода сотрудников.
- База данных сотрудников — актуальные сведения о сотрудниках и должностях.
- Табели учёта рабочего времени — отчёты, формируемые за произвольный период.
- Отчёты в формате CSV — файлы для передачи в бухгалтерию и кадровое подразделение.

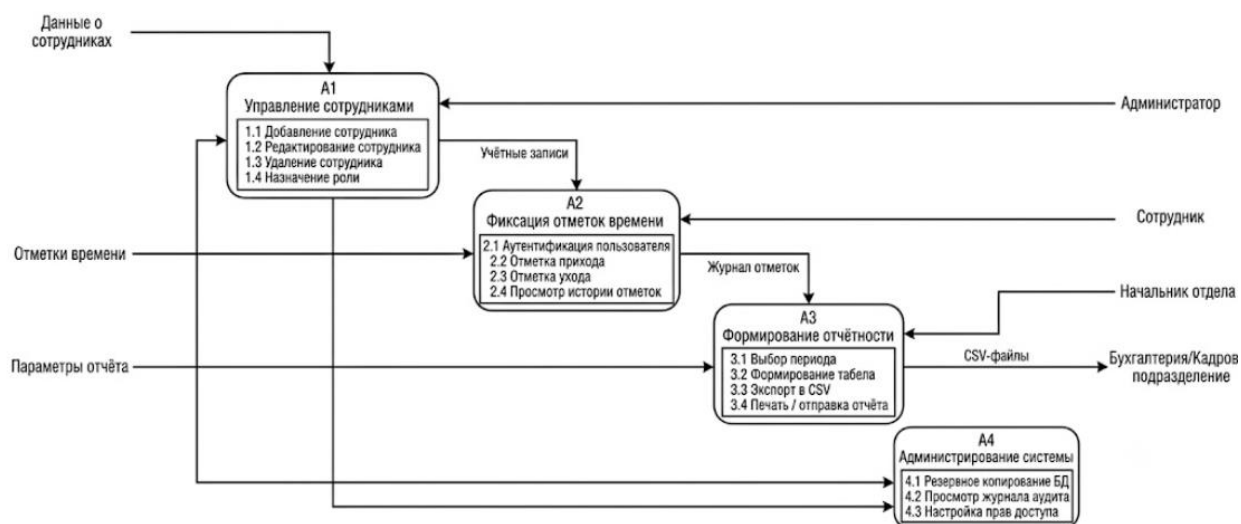


Рисунок 3. Декомпозиция процесса учёта рабочего времени IDEF0 A0

2.2. Моделирование процесса AS-IS

Модель AS-IS (как есть) отражает существующий порядок учёта рабочего времени сотрудников Ярославского гарнизонного военного суда до внедрения автоматизированной системы. В настоящее время учёт рабочего времени сотрудников аппарата суда осуществляется следующим образом:

Таблица 5. Этапы существующего процесса учёта рабочего времени

Этап	Действие	Ответственный	Способ выполнения	Временные затраты
1	Приход сотрудника на работу	Сотрудник	Ручная отметка в бумажном журнале	1-2 минуты
2	Фиксация времени прихода	Сотрудник	Запись ФИО, даты, времени	1-2 минуты
3	Рабочий день	Сотрудник	—	—
4	Уход с работы	Сотрудник	Ручная отметка в бумажном журнале	1-2 минуты
5	Фиксация времени ухода	Сотрудник	Запись ФИО, даты, времени	1-2 минуты
6	Ежемесячное формирование табеля	Начальник отдела	Ручной подсчёт по журналу	1-2 часа
7	Передача табеля в бухгалтерию	Начальник отдела	Бумажный документ	10-15 минут

Недостаток существующего подхода состоит в том, что информация о рабочем времени сотрудников хранится в бумажных журналах, которые могут быть утеряны или содержать ошибки. Кроме того, ручное формирование отчётности требует значительных временных затрат.

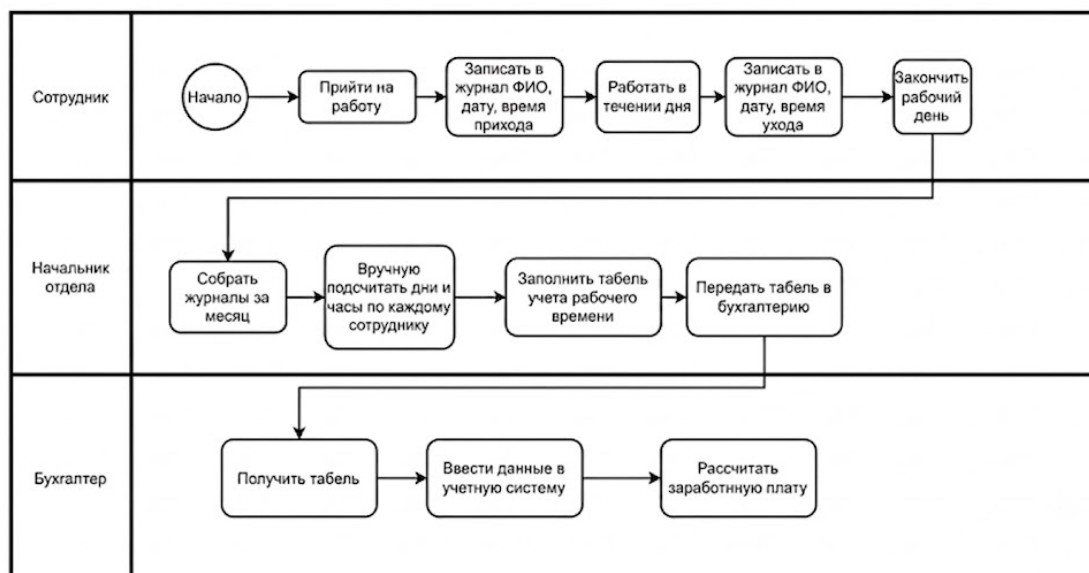


Рисунок 4. BPMN-модель процесса учёта рабочего времени AS-IS

2.3. Выявленные проблемы и ограничения

Таблица 6. Выявленные проблемы существующей организации учёта рабочего времени

Проблема	Проявление	Проектное решение
Ручное заполнение журнала	Сотрудник самостоятельно записывает время прихода/ухода	Автоматическая фиксация времени через приложение
Отсутствие контроля подлинности	Нет проверки, что запись сделана самим сотрудником	Аутентификация по логину и паролю
Разрозненное хранение данных	Журналы хранятся у начальника отдела	Единая база данных SQL Server
Трудоёмкость отчётности	Подсчёт отработанного времени выполняется вручную	Автоматическое формирование отчётов через SQL-запросы
Отсутствие оперативности	Руководитель не видит текущую явку сотрудников	Мгновенный просмотр отметок в таблице
Сложность анализа	Нет возможности быстро получить статистику	Формирование отчётов за произвольный период
Отсутствие журнала аудита	Нет информации об изменениях данных	Автоматическое журналирование операций

Выявленные проблемы подтверждают необходимость разработки автоматизированной системы учёта рабочего времени, в которой фиксация отметок, ведение базы сотрудников и формирование отчётности выполняются централизованно.

3. ОБОСНОВАНИЕ РАЗРАБОТКИ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

3.1. Цели автоматизации

Основная цель автоматизации — создание единой информационной системы для учёта рабочего времени сотрудников Ярославского гарнизонного военного суда, обеспечивающей автоматическую фиксацию времени прихода и ухода, централизованное хранение данных о сотрудниках и формирование отчётности за произвольный период.

Цели автоматизации формулируются таким образом, чтобы их можно было проверить в ходе тестирования: время отклика основных операций, корректность фиксации отметок, отсутствие ошибок при формировании отчётов, возможность экспорта в CSV и разграничение доступа пользователей.

Таблица 7. Цели автоматизации и показатели достижения

Цель	Показатель	Способ проверки
Сокращение времени на фиксацию отметок	Снижение времени отметки прихода/ухода до ≈ 10 секунд	Хронометраж операции до и после внедрения
Исключение ошибок ручного ввода	Отсутствие расхождений между фактическим и зафиксированным временем	Сверка с фактическим временем
Сокращение времени формирования отчётности	Формирование табеля за месяц не более чем за ≈ 10 секунд	Нагрузочная проверка на тестовых данных
Повышение объективности учёта	Невозможность изменения отметок самим сотрудником	Тесты безопасности, проверка прав доступа
Упрощение механизма штрафов и премий	Наличие объективных отчётов для обоснования начислений	Формирование отчёта за любой период
Централизация хранения данных	Единая база данных на сервере	Проверка целостности данных
Снижение совокупной стоимости владения	Собственная система дешевле типового решения	Сравнительный расчёт затрат

3.2. Анализ альтернативных решений

В качестве альтернативы собственной разработке можно рассмотреть следующие варианты:

Таблица 8. Сравнение вариантов автоматизации

Критерий	Типовое решение (1С:ЗУП)	Собственная разработка
Функциональность	Широкая, но избыточная для суда	Реализуются только нужные функции
Стоимость	Лицензия, внедрение, сопровождение	Основные затраты — время разработки
Адаптация	Требует доработок под специфику суда	Проектируется под процессы суда
Расширяемость	Ограничена возможностями платформы	Модульная архитектура, открытая структура БД
Срок внедрения	1-3 месяца	2 недели (учебный прототип)

Собственная система целесообразна в рамках практики, потому что позволяет реализовать минимально необходимое ядро, точно соответствующее индивидуальному заданию: регистрация сотрудников, фиксация отметок, отчётность, разграничение прав доступа.

3.3. Технико-экономическое обоснование

Технико-экономическое обоснование основано на сравнении трудозатрат при существующей и автоматизированной системе.

Таблица 9. Сравнение трудозатрат при выборе решения

Статья затрат	Существующая система	Автоматизированная система
Фиксация отметок в день (на 15 сотрудников)	15 чел. * (0,5 мин. приход + 0,5 мин. уход) = 15 мин. в день	15 чел. * (5 сек. приход + 5 сек. уход) = 150 сек. = 2,5 мин. в день
Формирование табеля за месяц	1 час	< 1 минуты
Передача данных в бухгалтерию	10 мин. (ручной ввод)	< 1 мин. (CSV-файл)

Дополнительный организационный эффект выражается в повышении прозрачности работы сотрудников и наличии объективной базы для принятия управленческих решений (премирование, дисциплинарные взыскания).

4. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА РАЗРАБОТКУ СИСТЕМЫ

4.1. Назначение и основания разработки

Разрабатываемая система имеет полное наименование «Автоматизированная система учёта рабочего времени сотрудников». Сокращенное обозначение — АСУ-УРВ. Объектом автоматизации являются процессы фиксации времени прихода и ухода сотрудников, ведения базы данных сотрудников, формирования отчётности.

Основаниями для разработки являются индивидуальное задание на производственную технологическую практику, программа практики по направлению 09.03.03 «Прикладная информатика», техническое задание на создание АСУ и потребность организации в повышении оперативности учёта рабочего времени.

Функциональное назначение — автоматизация процессов фиксации времени прихода и ухода сотрудников, ведения базы данных сотрудников, отделов и руководителей, формирования отчётности.

Эксплуатационное назначение — обеспечение круглосуточной (90% времени в год) бесперебойной работы системы учёта рабочего времени с графическим интерфейсом для администратора и сотрудников.

Таблица 10. Основные сведения о системе

Показатель	Значение
Полное наименование	Автоматизированная система учёта рабочего времени сотрудников
Сокращенное обозначение	АСУ-УРВ
Заказчик	Ярославский гарнизонный военный суд
Разработчик	Обучающийся МФЮА по направлению 09.03.03 «Прикладная информатика»
Платформа прототипа	C++/CLI, Visual Studio, SQL Server / SQL Server Management Studio
Основные пользователи	администратор, сотрудник

4.2. Функциональные требования

Функциональным требованиям присваиваются идентификаторы формата FR-XX. Такое обозначение позволяет связать требования с проектными решениями, тест-кейсами и фрагментами программного кода.

Таблица 11. Функциональные требования

№	Наименование функции	Результат выполнения	Временной регламент	Приоритет
FR-01	Аутентификация пользователя	Вход в систему с соответствующей ролью	≤ 3 сек	Высокий
FR-02	Фиксация прихода сотрудника	Запись в NoSQL с меткой времени	≤ 1 сек	Высокий
FR-03	Фиксация ухода сотрудника	Запись в NoSQL с меткой времени	≤ 1 сек	Высокий
FR-04	Добавление сотрудника (админ)	Новая запись в БД	≤ 2 сек	Средний
FR-05	Редактирование сотрудника (админ)	Обновление данных в БД	≤ 2 сек	Средний
FR-06	Удаление сотрудника (админ)	Удаление записи и каскадное удаление связанных данных	≤ 2 сек	Средний
FR-07	Формирование отчёта за период	CSV-файл с ФИО, должностью, временем входа/выхода	≤ 3 сек	Высокий
FR-08	Просмотр истории отметок (сотрудник)	Вывод списка отметок за день	≤ 2 сек	Низкий

4.3. Нефункциональные требования

Нефункциональные требования сформированы по модели качества программного продукта: функциональная пригодность, производительность, совместимость, удобство использования, надежность, защищенность, сопровождаемость и переносимость. Для каждого требования задан способ проверки.

Таблица 12. Функциональная пригодность

№	Подхарактеристика	Требование
NFR-01	Функциональная полнота	Система должна реализовывать все функции из п. 4.1 (FR-01 ... FR-08) без исключений.
NFR-02	Функциональная корректность	Результат выполнения каждой функции должен соответствовать описанию в п. 4.1 (например, FR-02 фиксирует именно время прихода, а не ухода).
NFR-03	Функциональная целостность	Система не должна допускать выполнения функций, противоречащих роли пользователя (сотрудник не может выполнять FR-04, FR-05, FR-06).

Таблица 13. Надёжность

№	Подхарактеристика	Требование
NFR-04	Защищённость от отказов	Система должна сохранять работоспособность не менее 90% времени в год.
NFR-05	Восстанавливаемость	Время восстановления после отказа (сбой питания, зависание) не более 10 минут.
NFR-06	Доступность	При плановом обслуживании (резервное копирование) система не отключается более чем на 5 минут.
NFR-07	Контроль входной информации	Использование регулярных выражений
NFR-08	Контроль выходной информации	Использование валидации данных

Таблица 14. Уровень производительности

№	Подхарактеристика	Требование
NFR-09	Временные характеристики	Время выполнения FR-02, FR-03 не более 1 сек; FR-07 не более 3 сек (при штатной нагрузке).
NFR-10	Использование ресурсов	Программа не должна потреблять более 200 МБ ОЗУ и 10% процессора в режиме ожидания.
NFR-11	Потенциальные возможности	Система должна сохранять отклик в указанных пределах при увеличении числа сотрудников до ≈ 500 записей в БД.

Таблица 15. Совместимость

№	Подхарактеристика	Требование
NFR-12	Сосуществование	Система должна корректно работать на одном ПК с MS Office, антивирусом и стандартным ПО суда без конфликтов.
NFR-13	Интероперабельность	Система должна выгружать отчёты (FR-07) в формате CSV, который открывается Excel и LibreOffice Calc без дополнительной обработки.

Таблица 16. Удобство использования

№	Подхарактеристика	Требование
NFR-14	Определимость пригодности	На главном экране должны быть понятные кнопки: «Приход», «Уход», «Отчёт».
NFR-15	Изучаемость	Сотрудник должен освоить базовые действия (вход, отметка прихода/ухода) за 5 минут после прочтения инструкции.
NFR-16	Управляемость	Все действия (добавление, удаление, формирование отчёта) доступны для администратора в отдельном для этого окне/вкладке.
NFR-17	Защищённость от ошибки пользователя	Перед удалением сотрудника (FR-06) система запрашивает подтверждение на действие.
NFR-18	Эстетика пользовательского интерфейса	Шрифты, отступы и цвета единообразны во всех окнах; окна не перегружены элементами.

Таблица 17. Защищённость

№	Подхарактеристика	Требование
NFR-19	Конфиденциальность	Пароли хранятся в хешированном виде (SHA-256).
NFR-20	Целостность	Данные о времени прихода/ухода (FR-02, FR-03) не могут быть изменены пользователем ни прямо, ни косвенно.
NFR-21	Доступность информации	Только администратор имеет доступ к FR-04, FR-05, FR-06. Сотрудник — только к FR-02, FR-03, FR-08.

Таблица 18. Сопровождаемость

№	Подхарактеристика	Требование
NFR-22	Возможность многократного использования	Функции работы с БД (вставка, обновление, удаление) реализованы как переиспользуемые методы в отдельном классе/классах.
NFR-23	Модифицируемость	Для изменения логики отчёта (FR-07) достаточно править один файл, не затрагивая другие модули.

Таблица 19. Переносимость

№	Подхарактеристика	Требование
NFR-24	Адаптируемость	Система работает на Windows 7/10/11 без изменения кода (при наличии .NET Framework 4.7.2).

Значения нефункциональных требований определены с учетом условий эксплуатации учебного прототипа в локальной сети организации. Для основных операций установлен предел в 1 секунду для FR-02 и FR-03, так как фиксация времени должна происходить практически мгновенно. Для FR-07 установлен предел в 3 секунды, так как формирование отчёта за период требует обработки большего объема данных.

4.4. Требования к данным, техническим средствам и безопасности

Входными данными являются учетные данные пользователей (логин, пароль), сведения о сотрудниках (ФИО, должность, логин, пароль, роль), отметки времени (приход/уход), параметры отчётного периода (дата начала, дата окончания). Выходными данными являются журнал отметок времени, база данных сотрудников, таблицы учёта рабочего времени, отчёты в формате CSV и сообщения об ошибках.

Минимальный состав технических средств включает сервер базы данных (SQL Server), рабочую станцию администратора, локальную сеть и резервное хранилище. Безопасность обеспечивается ролевым доступом (администратор / сотрудник), запросом пароля администратора для критических операций, внешними ключами, ограничениями целостности и регулярным резервным копированием.

Таблица 20. Входные и выходные данные системы

Тип данных	Состав реквизитов	Источник / приемник
Учетные данные	логин, пароль, роль, ФИО	форма входа / таблица Employee
Сотрудник	фамилия, имя, отчество, должность, логин, пароль, роль, дата приёма, активность	форма регистрации / таблица Employee
Отметка времени	тип отметки (приход/уход), дата и время, идентификатор сотрудника	форма отметок / таблица TimeEntry
Отчёт	период, ФИО, должность, дата, время прихода, время ухода	экран, CSV-файл

5. ПРОЕКТИРОВАНИЕ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

5.1. Модель процесса ТО-ВЕ

После внедрения автоматизированной системы учёта рабочего времени процесс фиксации прихода и ухода сотрудников становится централизованным и автоматизированным. Администратор (или начальник отдела) создаёт учётную запись сотрудника в системе, назначает ему роль (администратор / сотрудник). Сотрудник (или администратор) фиксирует время прихода и ухода через графический интерфейс приложения, система автоматически сохраняет отметки в базе данных с точной временной меткой.

Модель ТО-ВЕ устраняет дублирование данных, исключает ошибки ручного ввода и обеспечивает единый источник информации для всех пользователей. Руководитель может получать отчёты о рабочем времени без ручного подсчёта по бумажным журналам, а администратор может просматривать историю отметок любого сотрудника за любой период.

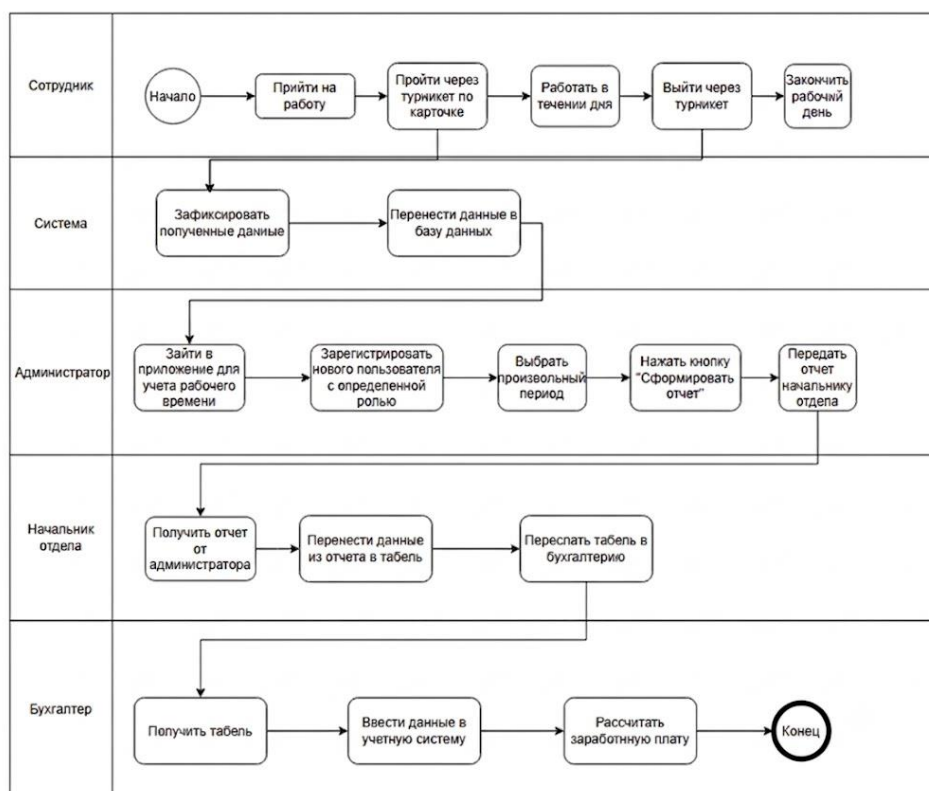


Рисунок 5. BPMN-модель процесса учёта рабочего времени ТО-ВЕ

5.2. Роли пользователей и варианты использования

В системе предусмотрены две основные роли: администратор и сотрудник. Роль определяет доступные функции и разделы интерфейса. Такой подход снижает риск несанкционированного изменения данных и соответствует требованию разграничения прав.

Диаграмма вариантов использования показывает, какие функции доступны пользователям. Администратор работает с сотрудниками (добавление, редактирование, удаление), фиксирует отметки времени для сотрудников (при необходимости) и формирует отчёты. Сотрудник имеет доступ только к просмотру своих отметок (истории).

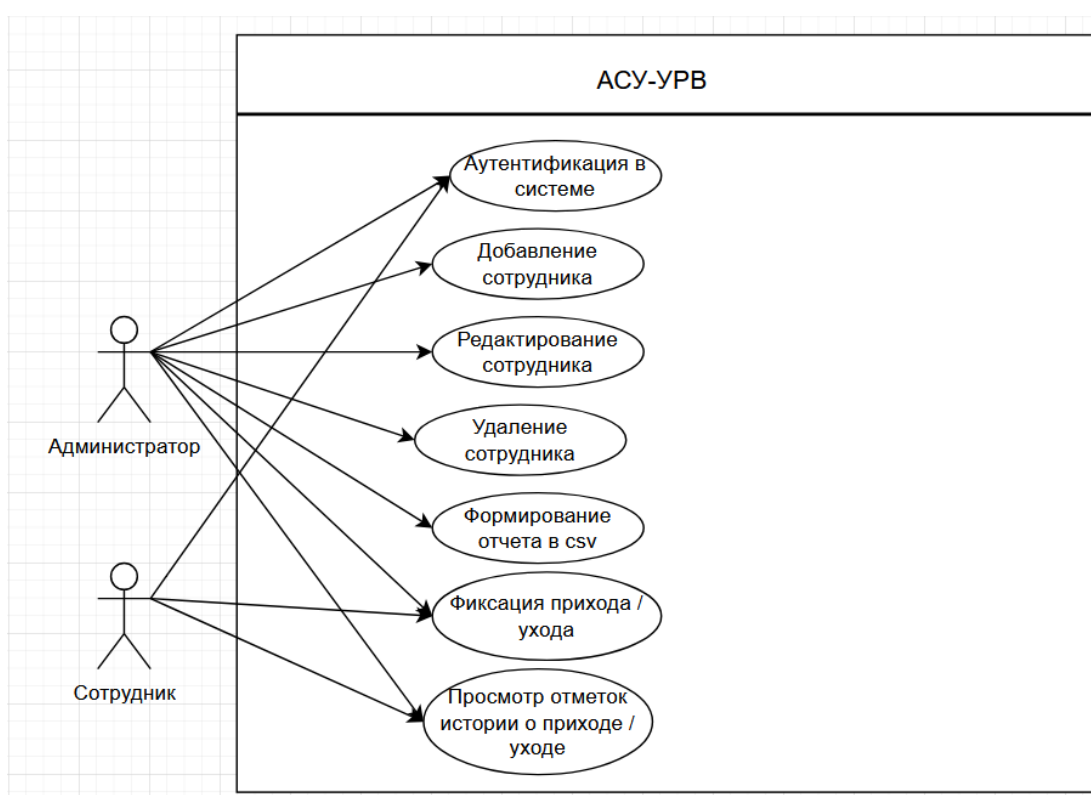


Рисунок 6. UML-диаграмма вариантов использования АСУ-УРВ

Таблица 21. Матрица доступа пользователей

Функция	Администратор	Сотрудник
Аутентификация в системе	+	+
Добавление сотрудника	+	—
Редактирование сотрудника	+	—

Продолжение таблицы 21. Матрица доступа пользователей

Удаление сотрудника	+	—
Фиксация прихода / ухода	+	+
Просмотр своих отметок	+	+
Просмотр отметок других сотрудников	+	—
Формирование отчёта за период	+	—

5.3. Архитектура системы и развертывание

Архитектура системы проектируется как клиент-серверная. Клиентское приложение, разработанное на C++/CLI в Visual Studio с использованием Windows Forms, выполняет проверку пользовательского ввода и обращается к базе данных через подключение ADO.NET (SqlClient). База данных SQL Server хранит справочники сотрудников, должностей, отметки времени и обеспечивает выполнение хранимых процедур.

В программной архитектуре выделены компоненты пользовательского интерфейса (формы), сервисов работы с данными, базы данных и хранимых процедур. Такое разделение позволяет сопровождать и тестировать модули независимо.

Для развертывания системы используется два компьютера в локальной сети организации:

- Серверный компьютер: установлен Microsoft SQL Server, создана база данных TimeTracking с таблицами Position, Employee, TimeEntry и хранимыми процедурами.
- Клиентский компьютер: установлено приложение employee_time_tracking.exe, которое подключается к серверному компьютеру через строку подключения с параметрами сервера, базы данных, логина и пароля.

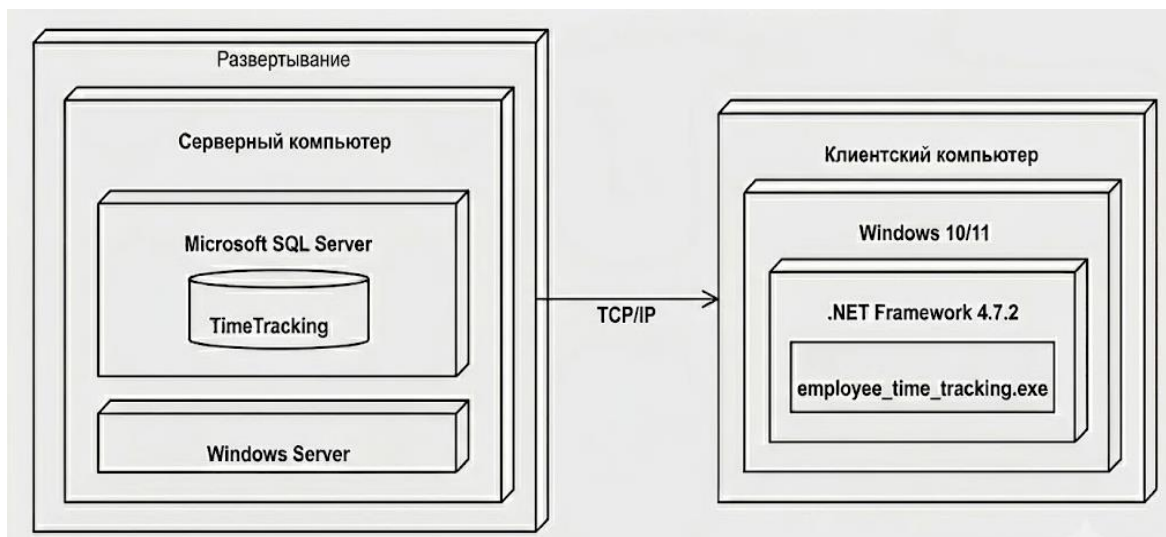


Рисунок 7. UML-диаграмма развертывания ACS-УРВ

5.4. Проектирование базы данных

База данных проектируется в реляционной модели. Основные таблицы отражают сущности предметной области: должности (Position), сотрудники (Employee) и отметки времени (TimeEntry). Для обеспечения целостности используются первичные и внешние ключи, ограничения CHECK, индексы и транзакции.

Ключевой принцип учёта рабочего времени — фиксация каждой отметки с типом CheckIn (приход) или CheckOut (уход). Время отметки сохраняется автоматически с помощью функции GETDATE() при вставке записи. Индекс на полях Employee_ID и Entry_Time обеспечивает быстрый поиск отметок по сотруднику и дате.

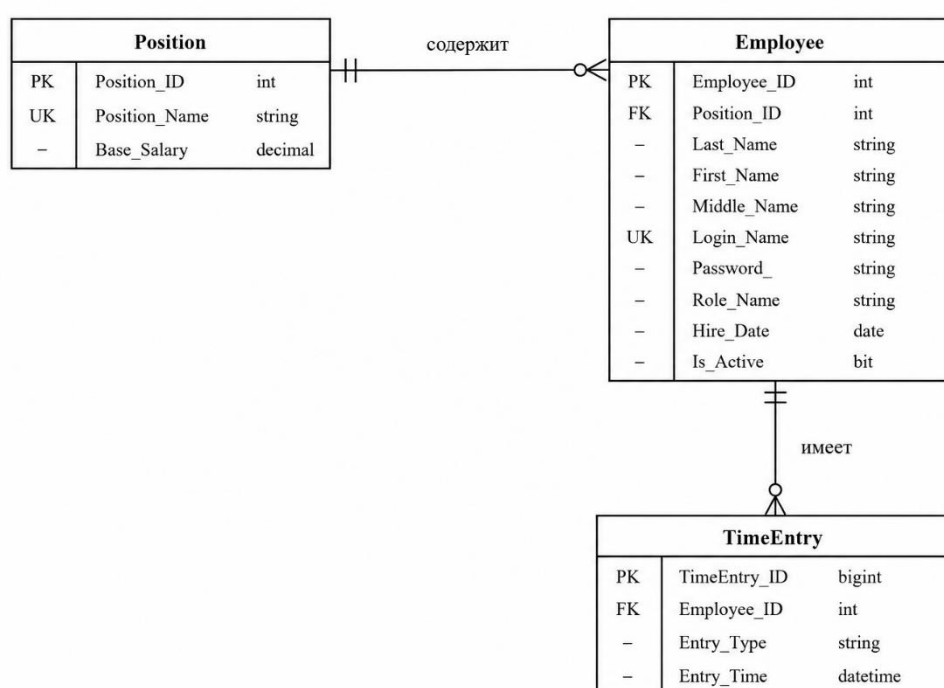


Рисунок 8. ER-диаграмма базы данных

Таблица Position (справочник должностей)

Назначение: хранение перечня должностей сотрудников.

Таблица 22. Структура таблицы Position

Поле	Тип	Null	Описание
Position_ID	INT	NOT NULL	Первичный ключ, автоинкремент (IDENTITY)
Position_Name	NVARCHAR(100)	NOT NULL	Наименование должности, уникальное

Ограничения:

- PRIMARY KEY (Position_ID)
- UNIQUE (Position_Name)

Таблица Employee (сотрудники)

Назначение: хранение информации о сотрудниках, их учётных данных и ролей.

Таблица 23. Структура таблицы Employee

Поле	Тип	Null	Описание
Employee_ID	INT	NOT NULL	Первичный ключ, автоинкремент (IDENTITY)
Last_Name	NVARCHAR(50)	NOT NULL	Фамилия
First_Name	NVARCHAR(50)	NOT NULL	Имя
Middle_Name	NVARCHAR(50)	NULL	Отчество
Position_ID	INT	NOT NULL	Внешний ключ к таблице Position
Login_Name	NVARCHAR(50)	NOT NULL	Логин для входа в систему, уникальный
Password_	NVARCHAR(255)	NOT NULL	Пароль (в учебном проекте без хеширования)
Role_Name	NVARCHAR(20)	NOT NULL	Роль: 'Admin' или 'Employee'
Hire_Date	DATE	NOT NULL	Дата приёма на работу (по умолчанию текущая)
Is_Active	BIT	NOT NULL	Активен ли сотрудник (1 — активен, 0 — уволен)

Ограничения:

- PRIMARY KEY (Employee_ID)
- FOREIGN KEY (Position_ID) REFERENCES Position (Position_ID)
- UNIQUE (Login_Name)
- CHECK (Role_Name IN ('Admin', 'Employee'))

Таблица TimeEntry (отметки времени)

Назначение: хранение отметок о приходе и уходе сотрудников.

Таблица 24. Структура таблицы TimeEntry

Поле	Тип	Null	Описание
TimeEntry_ID	BIGINT	NOT NULL	Первичный ключ, автоинкремент (IDENTITY)
Employee_ID	INT	NOT NULL	Внешний ключ к таблице Employee
Entry_Type	NVARCHAR(10)	NOT NULL	Тип отметки: 'CheckIn' (приход) или 'CheckOut' (уход)
Entry_Time	DATETIME	NOT NULL	Дата и время отметки (по умолчанию GETDATE())

Ограничения:

- PRIMARY KEY (TimeEntry_ID)
- FOREIGN KEY (Employee_ID) REFERENCES Employee (Employee_ID) ON DELETE CASCADE
- CHECK (Entry_Type IN ('CheckIn', 'CheckOut'))

Индексы:

- IX_TimeEntry_Employee_Time на полях (Employee_ID, Entry_Time) — для быстрого поиска отметок по сотруднику и дате

Хранимые процедуры

Для обеспечения безопасности и производительности все операции с базой данных вынесены в хранимые процедуры.

Таблица 25. Хранимые процедуры

Название	Назначение	Параметры
sp_AddEmployee	Добавление нового сотрудника	@LastName, @FirstName, @MiddleName, @PositionName, @LoginName, @PasswordHash, @UserRole

Продолжение таблицы 25. Хранимые процедуры

Название	Назначение	Параметры
sp_UpdateEmployee	Обновление данных сотрудника	@EmployeeID, @LastName, @FirstName, @MiddleName, @PositionName, @LoginName, @UserRole, @IsActive
sp_DeleteEmployee	Удаление сотрудника (с каскадным удалением отметок)	@EmployeeID
sp_AddTimeEntry	Добавление отметки времени	@EmployeeID, @EntryType
sp_AuthenticateUser	Аутентификация пользователя	@LoginName, @PasswordHash
sp_GetTimeReport	Формирование отчёта за период	@StartDate, @EndDate
sp_GetAllEmployees	Получение списка всех сотрудников	—
sp_GetTimeEntriesForPeriod	Получение отметок сотрудника за период	@EmployeeID, @StartDate, @EndDate
sp_GetLastTimeEntry	Получение последней отметки сотрудника	@EmployeeID
sp_GetAllPositions	Получение списка всех должностей	—

5.5. Проектирование классов и алгоритмов

Программа состоит из четырёх основных форм, каждая из которых отвечает за свою часть функционала.

Форма LoginForm предназначена для подключения к базе данных. Пользователь вводит имя сервера, название базы данных, логин и пароль. Также есть возможность использовать Windows-аутентификацию, при которой логин и пароль не требуются.

Форма RoleSelectionForm отвечает за выбор роли и вход в систему. Администратор может войти без ввода логина и пароля (по кнопке «Администратор»), а сотрудник должен ввести свой логин и пароль. При успешном входе программа запоминает роль пользователя.

Форма MainFormAdmin — это главное окно администратора. Оно разделено на три вкладки: «Управление сотрудниками», «Отметки времени» и «Формирование отчётов».

Форма RegisterForm используется для добавления новых сотрудников и редактирования данных существующих.

В программе также есть перечисление UserRole, которое определяет возможные роли: None (не выбрана), Admin (администратор) и Employee (сотрудник).

Все формы наследуются от стандартного класса Form, что позволяет им использовать базовые возможности оконного приложения.

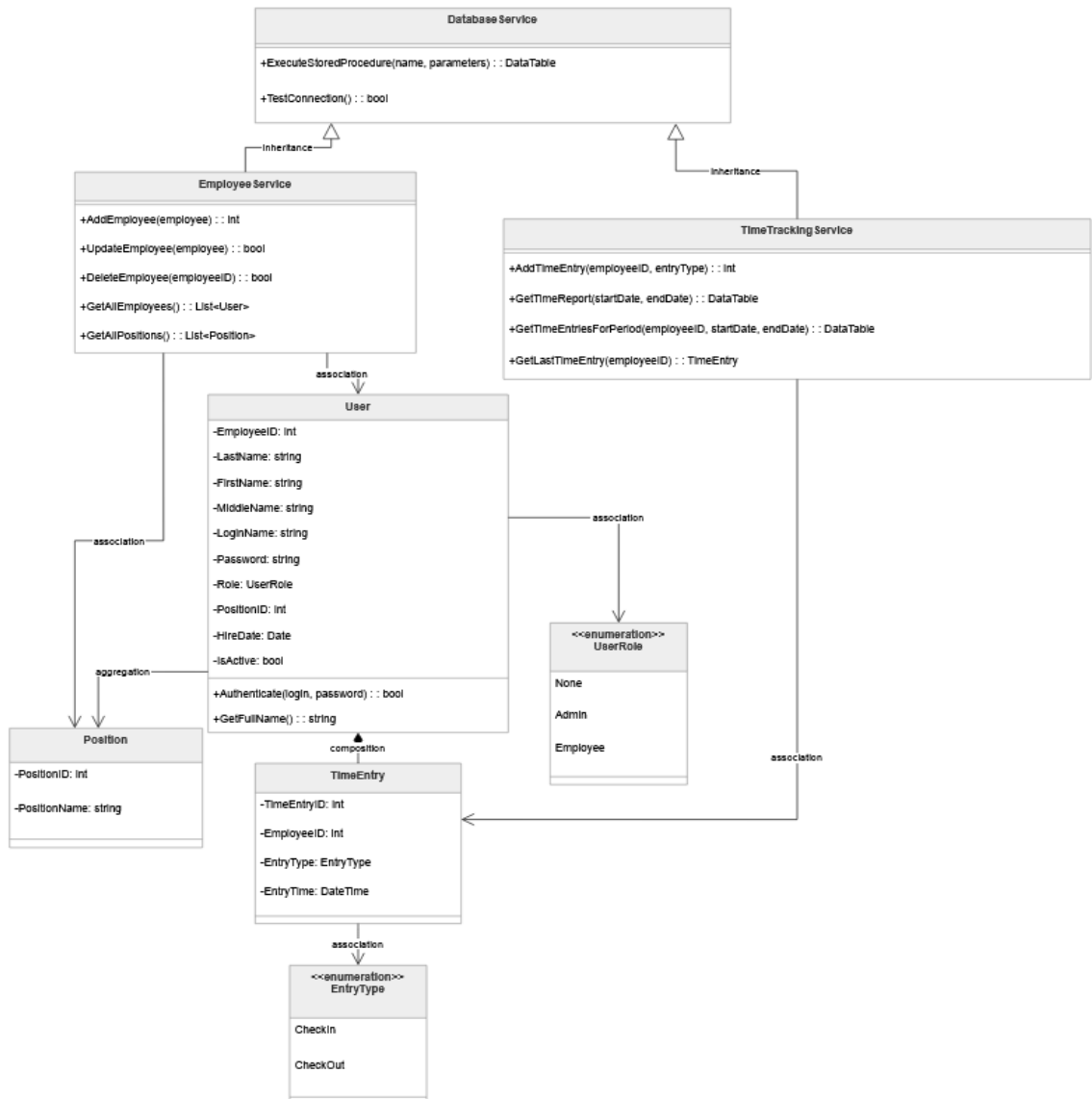


Рисунок 9. UML-диаграмма классов программного ядра

5.6. Жизненный цикл программы

Жизненный цикл программного ядра автоматизированной системы учёта рабочего времени сотрудников Ярославского гарнизонного военного суда рассмотрен с учетом положений следующих стандартов:

- ISO/IEC 33001:2015 — основные понятия оценки процессов.
- ISO/IEC 29148:2011 — требования к процессам разработки на протяжении жизненного цикла.
- ГОСТ Р ИСО/МЭК 15504-4-2012 — руководство по улучшению и оценке возможностей процессов.

В рамках учебного проекта жизненный цикл программы представлен как последовательность взаимосвязанных стадий, каждая из которых завершается получением конкретного результата: требований, проектных решений, программной реализации, результатов тестирования и эксплуатационной документации.

На стадии обоснования необходимости разработки были изучены существующие бизнес-процессы Ярославского гарнизонного военного суда, выявлены проблемы ручного учёта рабочего времени, дублирования данных в бумажных журналах, сложности контроля фактического присутствия сотрудников и трудоёмкости формирования табелей. Результатом данной стадии стало подтверждение необходимости создания единой автоматизированной системы учёта рабочего времени.

На стадии технического задания были определены назначение системы, функциональные требования (FR-01 ... FR-08) и нефункциональные требования (NFR-01 ... NFR-08), требования к данным (таблицы Position, Employee, TimeEntry), техническим средствам (серверный и клиентский компьютеры), безопасности и разграничению доступа (роли Admin и Employee). Данные результаты отражены в разделе 4 отчета и используются как основа для дальнейшего проектирования и тестирования.

На стадии эскизного и технического проектирования были разработаны модели процессов AS-IS и TO-BE, определены роли пользователей, варианты использования, архитектура системы (клиент-серверная), схема развертывания (сервер SQL Server + клиентское приложение), структура базы данных (таблицы Position, Employee, TimeEntry) и основные алгоритмы (аутентификация, фиксация отметки, формирование отчёта).

Данные решения позволили перейти от общей постановки задачи к конкретной проектной модели программного продукта.

На стадии рабочего проекта была выполнена реализация программного ядра на языке C++/CLI с использованием Microsoft SQL Server. В рамках реализации разработаны модули работы с сотрудниками (добавление, редактирование, удаление), фиксацией отметок времени (приход/уход), изменением ролей и формированием отчётов. Структура базы данных обеспечивает хранение справочника должностей, учётных записей сотрудников и отметок времени.

На стадии испытаний проведено функциональное тестирование реализованных требований FR-01...FR-08. Проверялись позитивные и негативные сценарии, граничные значения, корректность аутентификации, фиксации отметок, добавления, редактирования и удаления сотрудников, формирования отчётов и экспорта в CSV.

На стадии внедрения и сопровождения для системы предусматриваются установка программного продукта на клиентский компьютер, настройка подключения к SQL Server на серверном компьютере, создание учётных записей пользователей, назначение ролей, обучение пользователей и организация резервного копирования базы данных. В дальнейшем жизненный цикл может быть продолжен за счет реализации расширенной отчётности, журнала аудита, интеграции с кадровой и бухгалтерской системами.

6. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ЯДРА

6.1. Выбор инструментальных средств

Для реализации программного ядра автоматизированной системы учёта рабочего времени были выбраны следующие инструментальные средства:

Таблица 26. Инструментальные средства разработки

Средство	Назначение
Microsoft SQL Server	Хранение данных, выполнение хранимых процедур
SQL Server Management Studio (SSMS)	Создание и сопровождение схемы базы данных
Microsoft Visual Studio 2022	Разработка приложения на C++/CLI
C++/CLI	Реализация программного ядра и графического интерфейса
Windows Forms (.NET Framework 4.7.2)	Создание пользовательского интерфейса

6.2. Реализация структуры базы данных SQL Server

Структура базы данных разработана в реляционной модели. База данных TimeTracking включает три основные таблицы: Position (справочник должностей), Employee (сотрудники) и TimeEntry (отметки времени). Для обеспечения целостности данных используются первичные и внешние ключи, ограничения CHECK и индексы.

6.3. Реализация хранимых процедур

Все операции с базой данных вынесены в хранимые процедуры. Это обеспечивает безопасность (защита от SQL-инъекций) и централизацию бизнес-логики.

Таблица 27. Хранимые процедуры

Название	Назначение
sp_AddEmployee	Добавление нового сотрудника
sp_UpdateEmployee	Обновление данных сотрудника
sp_DeleteEmployee	Удаление сотрудника (с каскадным удалением отметок)
sp_AddTimeEntry	Добавление отметки времени (приход/уход)
sp_AuthenticateUser	Аутентификация пользователя
sp_GetTimeReport	Формирование отчёта за период
sp_GetAllEmployees	Получение списка всех сотрудников
sp_GetTimeEntriesForPeriod	Получение отметок сотрудника за период
sp_GetLastTimeEntry	Получение последней отметки сотрудника
sp_GetAllPositions	Получение списка всех должностей

6.4. Реализация C++/CLI-модулей в Visual Studio

Исходный код проекта структурирован по функциональным модулям. Проект включает 11 файлов:

Таблица 28. Состав файлов проекта

Файл	Назначение
all_includes.h	Общие подключения и using
LoginForm.h, LoginForm.cpp	Форма подключения к базе данных
Role.h, Role.cpp	Форма выбора роли и аутентификации
MainFormAdmin.h, MainFormAdmin.cpp	Панель администратора
RegisterForm.h, RegisterForm.cpp	Форма регистрации сотрудников
Main.h, Main.cpp	Точка входа в приложение

6.5. Связь требований, алгоритмов и реализации

Описание выбранных инструментальных средств, структуры базы данных, хранимых процедур, состава исходных файлов и ключевых алгоритмов, реализующих функциональные требования системы содержатся в таблице 29. Показана прослеживаемость от требований к конкретным программным модулям и хранимым процедурам.

Таблица 29. Связь требований, алгоритмов и реализации

Требование	Алгоритм / модуль	Реализация
FR-01	Аутентификация	sp_AuthenticateUser, RoleSelectionForm::AuthenticateUser
FR-02	Фиксация прихода	sp_AddTimeEntry с типом CheckIn
FR-03	Фиксация ухода	sp_AddTimeEntry с типом CheckOut
FR-04	Добавление сотрудника	sp_AddEmployee, RegisterForm::OnSaveClick
FR-05	Редактирование сотрудника	sp_UpdateEmployee, RegisterForm::OnSaveClick
FR-06	Удаление сотрудника	sp_DeleteEmployee, MainFormAdmin::OnDeleteClick
FR-07	Формирование отчёта	sp_GetTimeReport, MainFormAdmin::ExportToCsv
FR-08	Просмотр истории	sp_GetTimeEntriesForPeriod, MainFormAdmin::LoadTimeEntries

7. ТЕСТИРОВАНИЕ И ОТЛАДКА

7.1. Методы тестирования

Тестирование программного ядра проводилось с использованием следующих методов:

- Модульное тестирование — проверка каждого программного модуля (формы) на корректность работы в изолированной среде.
- Интеграционное тестирование — проверка взаимодействия модулей между собой и с базой данных.
- Функциональное тестирование — проверка соответствия реализованных функций требованиям технического задания.
- Тестирование граничных значений — проверка работы системы при вводе минимальных и максимальных допустимых значений.
- Негативное тестирование — проверка реакции системы на некорректные входные данные.

7.2. Тест-кейсы

В таблице 30 представлены тест-кейсы для проверки функциональных требований системы. Каждый тест-кейс содержит идентификатор, проверяемое требование, описание действия, ожидаемый результат и фактический результат.

Таблица 30. Тест-кейсы программного ядра

ID	Требование	Действие	Ожидаемый результат	Фактический результат	Статус
TC-01	FR-01	Ввод корректных логина и пароля сотрудника	Успешная аутентификация, открытие формы сотрудника	Успешная аутентификация, открытие формы сотрудника	Выполнено
TC-02	FR-01	Ввод некорректного пароля	Сообщение «Неверный логин или пароль»	Сообщение «Неверный пароль»	Выполнено
TC-03	FR-02	Выбор сотрудника и нажатие кнопки «Отметить приход»	Добавление записи в таблицу TimeEntry с типом CheckIn	Добавление записи в таблицу TimeEntry с типом CheckIn	Выполнено
TC-04	FR-03	Выбор сотрудника и нажатие кнопки «Отметить уход»	Добавление записи в таблицу TimeEntry с типом CheckOut	Добавление записи в таблицу TimeEntry с типом CheckOut	Выполнено
TC-05	FR-02, FR-03	Отметка прихода и ухода для одного сотрудника	Две записи в таблице TimeEntry с разными типами	Две записи в таблице TimeEntry с разными типами	Выполнено
TC-06	FR-04	Заполнение всех полей и нажатие «Зарегистрировать»	Добавление нового сотрудника в таблицу Employee	Добавление нового сотрудника в таблицу Employee	Выполнено
TC-07	FR-04	Оставление пустого обязательного поля	Сообщение об ошибке, запись не создаётся	Сообщение об ошибке, запись не создаётся	Выполнено

Продолжение таблицы 31. Тест-кейсы программного ядра

TC-08	FR-04	Ввод уже существующего логина	Сообщение «Логин уже существует»	Сообщение «Логин уже существует»	Выполнено
TC-09	FR-05	Выбор сотрудника, изменение данных и сохранение	Данные сотрудника обновлены в таблице Employee	Данные сотрудника обновлены в таблице Employee	Выполнено
TC-10	FR-05	Попытка редактирования без подтверждения пароля	Операция отклонена	Операция отклонена	Выполнено
TC-11	FR-06	Выбор сотрудника, подтверждение удаления	Сотрудник удалён, его отметки также удалены	Сотрудник удалён, его отметки также удалены	Выполнено
TC-12	FR-06	Отмена подтверждения удаления	Сотрудник и отметки остаются в БД	Сотрудник и отметки остаются в БД	Выполнено
TC-13	FR-07	Выбор периода (01.06.2026 – 30.06.2026) и нажатие «Сформировать отчёт»	Отображение таблицы с ФИО, должностью, датой, временем прихода/ухода	Отображение таблицы с ФИО, должностью, датой, временем прихода/ухода	Выполнено
TC-14	FR-07	Нажатие «Экспорт в CSV» после формирования отчёта	Создание CSV-файла в папке Reports	Создание CSV-файла в папке Reports	Выполнено
TC-15	FR-07	Формирование отчёта за период без данных	Пустая таблица, сообщение «Нет данных для экспорта»	Пустая таблица, сообщение «Нет данных для экспорта»	Выполнено
TC-16	FR-08	Просмотр истории отметок выбранного сотрудника	Отображение всех отметок сотрудника за выбранный период	Отображение всех отметок сотрудника за выбранный период	Выполнено

7.3. Результаты тестирования

По результатам тестирования установлено, что реализованное программное ядро выполняет основные функции, включённые в объём практической реализации. Позитивные сценарии по требованиям FR-02, FR-03, FR-04, FR-05, FR-06 и FR-07 выполнены успешно.

8. ОЦЕНКА КАЧЕСТВА ПРОГРАММЫ

8.1. Характеристики качества по ГОСТ ИСО/МЭК 25010-2026

Оценка качества разработанной автоматизированной системы учёта рабочего времени выполнена в соответствии с ГОСТ Р ИСО/МЭК 25010-2015 «Информационная технология. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения. Модели качества систем и программных продуктов».

Согласно стандарту, выделены следующие характеристики качества с оценкой их достижения в рамках учебного проекта:

Таблица 32. Оценка характеристик качества программного продукта

Характеристика	Подхарактеристика	Оценка	Обоснование
Функциональная пригодность	Функциональная полнота	Высокая	Реализованы все требования FR-01...FR-08, за исключением пунктов, оставленных на дальнейшее развитие
	Функциональная корректность	Средняя	Базовые сценарии работают корректно, отдельные проверки требуют доработки
	Функциональная целесообразность	Высокая	Система соответствует потребностям Ярославского гарнизонного военного суда
Надёжность	Защищённость от отказов	Средняя	Система сохраняет работоспособность при штатных сценариях
	Восстанавливаемость	Средняя	Восстановление возможно через перезапуск приложения и проверку подключения к БД
	Отказоустойчивость	Низкая	Учебный прототип не предусматривает кластеризацию и резервирование

Продолжение таблицы 33. Оценка характеристик качества программного продукта

Производительность	Временная характеристика	Высокая	Отметка прихода/ухода — менее 1 секунды, формирование отчёта — менее 3 секунд
	Использование ресурсов	Высокая	Потребление ОЗУ менее 200 МБ, загрузка ЦП менее 10% в режиме ожидания
	Пропускная способность	Не требуется	Система рассчитана на 15–50 сотрудников, нагрузка низкая
Удобство использования	Определимость пригодности	Высокая	Интерфейс содержит понятные кнопки и подписи на русском языке
	Изучаемость	Высокая	Базовые действия осваиваются за 5 минут
	Управляемость	Высокая	Все функции администратора доступны в отдельных вкладках
	Защищённость от ошибки пользователя	Средняя	Присутствует валидация с регулярными выражениями, подтверждение удаления
	Эстетика интерфейса	Средняя	Единообразие шрифтов и цветов, окна не перегружены элементами

Продолжение таблицы 34. Оценка характеристик качества программного продукта

Безопасность	Конфиденциальность	Низкая	Пароли хранятся без хеширования (учебный проект)
	Целостность	Средняя	Данные об отметках не могут быть изменены пользователем
	Доступность информации	Высокая	Разграничение прав доступа: администратор / сотрудник
Сопровождаемость	Модульность	Высокая	Код разделён на 12 логических файлов, каждая форма в отдельном модуле
	Переиспользуемость	Средняя	Хранимые процедуры могут быть переиспользованы в других проектах
	Анализируемость	Средняя	Код имеет комментарии, названия переменных отражают их назначение
	Модифицируемость	Высокая	Для изменения логики отчёта достаточно править один файл
Переносимость	Адаптируемость	Высокая	Система работает на Windows 7/10/11 без изменения кода
	Устанавливаемость	Высокая	Распространяется в виде ZIP-архива с инструкцией
	Заменяемость	Низкая	Полная замена возможна только после разработки новой версии

ЗАКЛЮЧЕНИЕ

По итогам прохождения производственной технологической практики были освоены и закреплены компетенции, формируемые в рамках практической подготовки.

В результате прохождения практики обучающийся должен знать основные принципы анализа предметной области, методы описания бизнес-процессов, основы проектирования информационных систем, правила разработки структуры базы данных, требования к оформлению проектной и отчетной документации, а также основные подходы к тестированию программного продукта.

В ходе практики были сформированы умения анализировать существующие процессы организации, выявлять недостатки в работе информационной системы или учетного процесса, формулировать функциональные и нефункциональные требования к программному продукту, разрабатывать модели бизнес-процессов AS-IS и TO-BE, проектировать структуру базы данных, создавать диаграммы и подготавливать материалы для технической документации.

По результатам практики обучающийся должен владеть навыками построения IDEF0-, BPMN-, UML- и ER-диаграмм, навыками проектирования базы данных, разработки программной логики, проведения тестирования, анализа полученных результатов и подготовки отчета в соответствии с требованиями ГОСТов и методических указаний.

В ходе производственной технологической практики была изучена деятельность организации, проанализирован существующий процесс учета и обработки данных, а также выявлены основные проблемы действующего подхода: дублирование информации, высокая трудоемкость ручных операций, риск возникновения ошибок, недостаточная прозрачность действий пользователей и отсутствие удобного автоматизированного контроля.

В период практики были выполнены следующие виды работ: проведен анализ предметной области, определены цели и задачи автоматизации, сформированы требования к разрабатываемой системе, подготовлены модели текущего и проектируемого процессов, разработана структура базы данных, описана логика работы программного продукта, а также подготовлены тест-кейсы для проверки корректности реализованных функций.

На основании проведенного анализа был разработан проект автоматизированной информационной системы, направленной на повышение эффективности учета, сокращение

времени выполнения операций и снижение вероятности ошибок при работе с данными. В отчет включены IDEF0-диаграммы, BPMN-модели AS-IS и TO-BE, UML-диаграммы, ER-диаграмма базы данных, таблицы требований, описание алгоритмов работы системы и результаты тестирования.

В рамках реализации была подготовлена структура базы данных Microsoft SQL Server, включающая основные таблицы, связи между ними и ограничения целостности данных. Программная часть системы реализована с использованием языка C++/C++/CLI в среде Visual Studio. Разработанное решение обеспечивает выполнение основных функций учета, поиска, обработки и хранения информации, а также формирование необходимых отчетных данных.

Проведенное тестирование подтвердило корректность основной реализованной логики. Система выполняет предусмотренные операции, обеспечивает обработку данных в соответствии с заданными требованиями и позволяет снизить трудоемкость выполнения ручных действий. Выявленные в ходе работы ограничения учебного прототипа не противоречат цели практики, так как часть требований может быть реализована в дальнейшем при развитии программного продукта до уровня промышленного решения.

В процессе подготовки и написания отчета были учтены вопросы анализа предметной области, моделирования бизнес-процессов, проектирования базы данных, разработки программного обеспечения, тестирования функций системы, оценки эффективности предложенного решения и соблюдения требований к оформлению отчетной документации.

Таким образом, цели и задачи производственной технологической практики были достигнуты. В ходе практики были закреплены теоретические знания, получены практические навыки проектирования и разработки информационной системы, а также сформированы умения анализировать деятельность организации и предлагать обоснованные решения для автоматизации рабочих процессов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Нормативные правовые акты и стандарты

1. ГОСТ 19.101 77 Единая система программной документации. Виды программ и программных документов. — Введ. 1980 01 01. — Москва : Стандартинформ, 2010. — 8 с.
2. ГОСТ 19.201 78 Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — Введ. 1980 01 01. — Москва : Издательство стандартов, 1986. — 9 с.
3. ГОСТ 19.301 79 Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. — Введ. 1980 01 01. — Москва : Издательство стандартов, 1986. — 7 с.
4. ГОСТ 19.402 78 Единая система программной документации. Описание программы. — Введ. 1980 01 01. — Москва : Издательство стандартов, 1986. — 10 с.
5. ГОСТ 19.404 79 Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — Введ. 1981 01 01. — Москва : Издательство стандартов, 1986. — 6 с.
6. ГОСТ 19.505 79 Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению. — Введ. 1980 01 01. — Москва : Издательство стандартов, 1986. — 5 с.
7. ГОСТ 19.701 90 (ИСО 5807 85) Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. — Введ. 1992 01 01. — Москва : Издательство стандартов, 1991. — 25 с.
8. ГОСТ 34.602 2020 Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — Введ. 2022 01 01. — Москва : Российский институт стандартизации, 2022. — 24 с.
9. ГОСТ Р 7.0.100 2018 Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Введ. 2019 07 01. — Москва : Стандартинформ, 2018. — 98 с.
10. ГОСТ Р ИСО/МЭК 12207 2016 Информационные технологии. Системная и программная инженерия. Процессы жизненного цикла программных средств. — Введ. 2017 11 01. — Москва : Стандартинформ, 2017. — 108 с.

11. ГОСТ Р ИСО/МЭК 25010 2026 Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества. — Введ. 2026 07 01 (предположительно). — Москва : Стандартинформ, 2026. — 29 с.

Литература (книги, учебные пособия)

12. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; перевод с английского А. Киселева. — Санкт Петербург : Питер, 2022. — 350 с. — (Библиотека программиста). — ISBN 978 5 4461 0772 8.
13. Петкович, Д. Microsoft SQL Server 2019. Руководство для начинающих / Д. Петкович. — 7 е изд. — New York : McGraw Hill Education, 2020. — 861 p. — ISBN 978 1 260 45592 9.
14. Троелсен, Э. Язык программирования C# 9 и платформа .NET 5. Основные принципы и практики программирования / Э. Троелсен, Ф. Джепикс. — 10 е изд. — Москва : Диалектика, 2022. — 1392 с. — ISBN 978 5 907458 67 3.
15. Фарли, Д. Современная программная инженерия. ПО в эпоху эджайла и непрерывного развертывания / Д. Фарли ; перевод с английского М. Трусковской. — Санкт Петербург [и др.] : Питер, 2024. — 283 с. — (Для профессионалов). — ISBN 978 5 4461 2335 3.

Электронные ресурсы и прочая документация

16. C++/CLI and Windows Forms: [сайт]. — Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/> (дата обращения: 10.06.2026).
17. BPMN.io – инструмент для моделирования бизнес-процессов : [сайт]. — Режим доступа: <https://bpmn.io/> (дата обращения: 13.06.2026)
18. diagrams.net – онлайн-редактор диаграмм : [сайт]. — Режим доступа: <https://app.diagrams.net/#>
19. ISO 25010:2026 Systems and software Quality Requirements and Evaluation (SQuaRE) : продуктовая модель качества. — Geneva : ISO, 2026.
20. СТО МФЮА 04 2025. Производственная практика (технологическая (проектно технологическая)). Методические указания по выполнению. — Москва : МФЮА, 2025. — 35 с.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А. Руководство пользователя

1. Назначение программы

Автоматизированная система учёта рабочего времени сотрудников (далее — Система) предназначена для автоматизации процесса фиксации времени прихода и ухода сотрудников Ярославского гарнизонного военного суда, ведения единой базы данных сотрудников и должностей, а также формирования отчётности за произвольный период.

Система используется администратором суда для управления учётными записями сотрудников и начальником отдела для контроля явки и формирования таблиц рабочего времени.

2. Требования к техническому обеспечению

Для работы Системы необходимо следующее техническое обеспечение:

Таблица 1. Требования к техническому обеспечению

Компонент	Минимальные требования
Процессор	Intel Core i3 / AMD Ryzen 3
Оперативная память	4 ГБ
Свободное дисковое пространство	1 ГБ
Операционная система	Windows 7 SP1 / Windows 10 / Windows 11
Дополнительное ПО	.NET Framework 4.7.2, MS SQL Server (для серверной части)
Сеть	Локальная сеть для подключения к серверу БД

3. Установка и настройка программы

3.1. Установка

1. Скопируйте папку с программой employee_time_tracking на клиентский компьютер.
2. Запустите файл employee_time_tracking.exe.

3.2. Настройка подключения к базе данных

Перед началом работы необходимо настроить подключение к серверу баз данных:

1. Запустите программу. Откроется форма подключения к базе данных.
2. В поле «Сервер» введите имя сервера SQL Server (по умолчанию: WIN-289CA8726EM).
3. В поле «База данных» введите имя базы данных (по умолчанию: TimeTracking).
4. Выберите тип аутентификации:
 - **Windows аутентификация** — используется учётная запись Windows (поля логина и пароля отключаются).
 - **SQL Server аутентификация** — требуется ввести логин и пароль пользователя SQL Server (например, sa).
5. Нажмите кнопку «Подключиться».
6. При успешном подключении появится сообщение «Подключение к базе данных успешно установлено!».

4. Работа с программой

4.1. Вход в систему

После успешного подключения к базе данных откроется форма выбора роли пользователя.

Для входа как администратор:

1. Нажмите кнопку «Администратор». Администратор входит в систему без проверки логина и пароля.

Для входа как сотрудник:

1. Введите логин и пароль, полученные от администратора.
2. Нажмите кнопку «Войти».
3. При успешной аутентификации откроется панель сотрудника (в текущей версии реализована частично).

4.2. Панель администратора

Панель администратора содержит три вкладки: «Управление сотрудниками», «Отметки времени» и «Формирование отчётов».

4.2.1. Вкладка «Управление сотрудниками»

Вкладка предназначена для ведения базы данных сотрудников.

Просмотр списка сотрудников:

Список всех сотрудников отображается в таблице со следующими колонками:

- ID — уникальный идентификатор сотрудника
- Фамилия
- Имя
- Отчество
- Должность
- Логин
- Роль (Admin / Employee)
- Дата приёма
- Активен (Да / Нет)

Добавление сотрудника:

1. Нажмите кнопку «Добавить сотрудника».
2. В появившемся окне введите пароль администратора (по умолчанию: admin123).
3. Заполните все поля формы регистрации:
 - **Фамилия** — обязательно, только русские буквы, первая заглавная, допускается дефис (например: Иванов, Убей-волк)
 - **Имя** — обязательно, только русские буквы, первая заглавная, допускается дефис (например: Анна-Мария)
 - **Отчество** — необязательно, только русские буквы, первая заглавная
 - **Должность** — выбор из списка
 - **Логин** — обязательно, только латинские буквы, цифры и подчёркивание, от 3 до 50 символов
 - **Пароль** — обязательно, от 4 до 255 любых символов
 - **Подтверждение пароля** — должно совпадать с паролем

- **Роль** — выбор: Employee (сотрудник) или Admin (администратор)

4. Нажмите кнопку «Зарегистрировать».
5. При успешной регистрации появится сообщение с ID нового сотрудника.

Редактирование сотрудника:

1. Выберите сотрудника в таблице (щёлкните по строке).
2. Нажмите кнопку «Редактировать».
3. Введите пароль администратора.
4. Внесите необходимые изменения в форму редактирования.
5. Нажмите кнопку «Сохранить».
6. При редактировании пароль не изменяется.

Удаление сотрудника:

1. Выберите сотрудника в таблице (щёлкните по строке).
2. Нажмите кнопку «Удалить».
3. Введите пароль администратора.
4. Подтвердите удаление в диалоговом окне.
5. При удалении сотрудника все его отметки времени удаляются автоматически.

Обновление списка:

Нажмите кнопку «Обновить» для перезагрузки списка сотрудников.

4.2.2. Вкладка «Отметки времени»

Вкладка предназначена для фиксации времени прихода и ухода сотрудников, а также просмотра истории отметок.

Выбор сотрудника:

В выпадающем списке «Сотрудник» выберите нужного сотрудника.

Установка периода отображения:

- «С даты» — начальная дата периода
- «По дате» — конечная дата периода
- Нажмите кнопку «Показать» для применения фильтра

Фиксация отметки прихода:

1. Выберите сотрудника в выпадающем списке.
2. Нажмите кнопку «Отметить приход».
3. Система автоматически фиксирует текущую дату и время.

4. Появится сообщение «Отметка о приходе добавлена!».

Фиксация отметки ухода:

1. Выберите сотрудника в выпадающем списке.
2. Нажмите кнопку «Отметить уход».
3. Система автоматически фиксирует текущую дату и время.
4. Появится сообщение «Отметка об уходе добавлена!».

Просмотр истории отметок:

Таблица отображает все отметки выбранного сотрудника за указанный период:

- Тип (приход / уход)
- Дата
- Время

Последняя отметка:

В верхней части вкладки отображается информация о последней отметке выбранного сотрудника с указанием типа (ПРИХОД / УХОД) и времени.

4.2.3. Вкладка «Формирование отчётов»

Вкладка предназначена для формирования табелей учёта рабочего времени.

Формирование отчёта:

1. Выберите начальную дату периода в поле «Начало периода».
2. Выберите конечную дату периода в поле «Конец периода».
3. Нажмите кнопку «Сформировать отчёт».
4. В таблице отобразится отчёт со следующими колонками:
 - ID сотрудника
 - ФИО сотрудника
 - Должность
 - Дата
 - Время прихода
 - Время ухода

Экспорт в CSV:

1. После формирования отчёта нажмите кнопку «Экспорт в CSV».
2. Система автоматически создаст папку Reports в директории с программой.
3. Файл сохранится с именем в формате: Report_ГГГГ-ММ-ДД_ЧЧ-мм-сс.csv.

4. Появится сообщение об успешном экспорте с указанием пути к файлу.

5. Завершение работы

Для завершения работы с программой:

1. Закройте главное окно приложения (крестик в правом верхнем углу).
2. Система автоматически завершит все соединения с базой данных.
3. При необходимости сделайте резервную копию базы данных.

6. Резервное копирование

Рекомендуется регулярно выполнять резервное копирование базы данных с помощью средств SQL Server.

Команда для создания резервной копии:

```
EXEC xp_create_subdir 'C:\employee_time_tracking\Backup\TimeTracking.bak'
```

```
BACKUP DATABASE TimeTracking TO DISK = 'C:\employee_time_tracking\Backup\TimeTracking.bak'
```

Периодичность резервного копирования: **еженедельно**. Ответственный за резервное копирование — администратор суда.

Приложение Б. Листинг файла all_includes.h

```
#pragma once

#using<System.dll>
#using<System.Data.dll>
#using<System.Windows.Forms.dll>
#using<System.Drawing.dll>
#using<System.Reflection.dll>

#include<Windows.h>
#include<sql.h>
#include<sqlext.h>
#include<sqltypes.h>
#include<vcclr.h>

using namespace System;
using namespace System::ComponentModel;
using namespace System::Collections;
using namespace System::Windows::Forms;
using namespace System::Data;
using namespace System::Data::SqlClient;
using namespace System::Drawing;
using namespace System::Text::RegularExpressions;
using namespace System::Management;
using namespace System::IO;
using namespace System::Diagnostics;
using namespace System::Reflection;
```

Приложение В. Листинг файла LoginForm.h

```
#pragma once
#include "../include/all_includes.h"

public ref class LoginForm : public Form
{
private:
    TextBox^ serverTextBox;
    TextBox^ databaseTextBox;
    TextBox^ usernameTextBox;
    TextBox^ passwordTextBox;
    CheckBox^ integratedAuthCheckBox;
    Button^ connectButton;

    SqlConnection^ sqlConnection;
    bool is_connected;

    void InitializeComponent();
    void integratedAuthCheckBox_CheckedChanged(Object^ sender, EventArgs^
args);
    void connectButton_Click(Object^ sender, EventArgs^ args);

public:
    LoginForm();

    property bool IsConnected
    {
        bool get() { return is_connected; }
    }

    property SqlConnection^ Connection
    {
        SqlConnection^ get() { return sqlConnection; }
    }
};
```

Приложение Г. Листинг файла LoginForm.cpp

```

#include "../include/LoginForm.h"
#include "../include/RegisterForm.h"

LoginForm::LoginForm()
{
    is_connected = false;
    InitializeComponent();
}

void LoginForm::InitializeComponent()
{
    this->Text = gcnew System::String(L"Система учёта рабочего времени -
Подключение к базе данных");
    this->Size = System::Drawing::Size(450, 350);
    this->StartPosition = FormStartPosition::CenterScreen;
    this->FormBorderStyle =
System::Windows::Forms::FormBorderStyle::FixedDialog;
    this->MaximizeBox = false;

    Label^ titleLabel = gcnew Label();
    titleLabel->Text = gcnew System::String(L"Система учёта рабочего
времени");
    titleLabel->SetBounds(20, 10, 400, 25);
    titleLabel->Font = gcnew System::Drawing::Font(L"Segoe UI", 12,
FontStyle::Bold);
    titleLabel->TextAlign = ContentAlignment::MiddleCenter;
    this->Controls->Add(titleLabel);

    Label^ serverLabel = gcnew Label();
    serverLabel->Text = gcnew System::String(L"Сервер:");
    serverLabel->SetBounds(20, 50, 100, 20);
    this->Controls->Add(serverLabel);

    serverTextBox = gcnew TextBox();
    serverTextBox->SetBounds(130, 50, 280, 20);
    serverTextBox->Text = L"WIN-289CA8726EM";
    this->Controls->Add(serverTextBox);

    Label^ databaseLabel = gcnew Label();
    databaseLabel->Text = gcnew System::String(L"База данных:");
    databaseLabel->SetBounds(20, 80, 100, 20);
    this->Controls->Add(databaseLabel);

    databaseTextBox = gcnew TextBox();

```

```

databaseTextBox->SetBounds(130, 80, 280, 20);
databaseTextBox->Text = L"TimeTracking";
this->Controls->Add(databaseTextBox);

integratedAuthCheckBox = gcnew CheckBox();
integratedAuthCheckBox->Text = gcnew System::String(L"Windows
аутентификация");
integratedAuthCheckBox->SetBounds(20, 110, 200, 20);
integratedAuthCheckBox->Checked = true;
integratedAuthCheckBox->CheckedChanged += gcnew EventHandler(this,
&LoginForm::integratedAuthCheckBox_CheckedChanged);
this->Controls->Add(integratedAuthCheckBox);

Label^ usernameLabel = gcnew Label();
usernameLabel->Text = gcnew System::String(L"Пользователь:");
usernameLabel->SetBounds(20, 140, 100, 20);
this->Controls->Add(usernameLabel);

usernameTextBox = gcnew TextBox();
usernameTextBox->SetBounds(130, 140, 280, 20);
usernameTextBox->Enabled = false;
this->Controls->Add(usernameTextBox);

Label^ passwordLabel = gcnew Label();
passwordLabel->Text = gcnew System::String(L"Пароль:");
passwordLabel->SetBounds(20, 170, 100, 20);
this->Controls->Add(passwordLabel);

passwordTextBox = gcnew TextBox();
passwordTextBox->SetBounds(130, 170, 280, 20);
passwordTextBox->PasswordChar = '*';
passwordTextBox->Enabled = false;
this->Controls->Add(passwordTextBox);

connectButton = gcnew Button();
connectButton->Text = gcnew System::String(L"Подключиться");
connectButton->SetBounds(165, 220, 120, 35);
connectButton->BackColor = Color::LightBlue;
connectButton->FlatStyle = FlatStyle::Flat;
connectButton->Click += gcnew EventHandler(this,
&LoginForm::connectButton_Click);
this->Controls->Add(connectButton);

Label^ infoLabel = gcnew Label();
infoLabel->Text = gcnew System::String(L"Подключитесь к базе данных
для доступа к системе");
infoLabel->SetBounds(20, 270, 400, 40);

```



```

        infoLabel->TextAlign = ContentAlignment::MiddleCenter;
        infoLabel->ForeColor = Color::Gray;
        this->Controls->Add(infoLabel);
    }

    void LoginForm::integratedAuthCheckBox_CheckedChanged(Object^ sender,
    EventArgs^ args)
    {
        bool isEnabled = !integratedAuthCheckBox->Checked;
        usernameTextBox->Enabled = isEnabled;
        passwordTextBox->Enabled = isEnabled;
    }

    void LoginForm::connectButton_Click(Object^ sender, EventArgs^ args)
    {
        String^ connectionString;

        if (integratedAuthCheckBox->Checked)
        {
            connectionString = String::Format(
                L"Data Source={0};Initial Catalog={1};Integrated
Security=True;Connect Timeout=30;",
                serverTextBox->Text, databaseTextBox->Text);
        }
        else
        {
            connectionString = String::Format(
                L"Data Source={0};Initial Catalog={1};User
ID={2};Password={3};Connect Timeout=30;",
                serverTextBox->Text, databaseTextBox->Text, usernameTextBox-
>Text, passwordTextBox->Text);
        }

        try
        {
            sqlConnection = gcnew SqlConnection(connectionString);
            sqlConnection->Open();

            SqlCommand^ checkCommand = gcnew SqlCommand(
                L"SELECT COUNT(*) FROM INFORMATION_SCHEMA.TABLES WHERE
TABLE_NAME = 'Employee'",
                sqlConnection);
            int tableCount = Convert::ToInt32(checkCommand->ExecuteScalar());

            if (tableCount == 0)
            {
                MessageBox::Show(

```

```

        gcnew System::String(L"База данных не содержит
необходимых таблиц.\nВыполните скрипт TimeTracking.sql"),
        gcnew System::String(L"Предупреждение"),
        MessageBoxButtons::OK,
        MessageBoxIcon::Warning);
    sqlConnection->Close();
    return;
}

is_connected = true;

MessageBox::Show(
    gcnew System::String(L"Подключение к базе данных успешно
установлено!"),
    gcnew System::String(L"Успех"),
    MessageBoxButtons::OK,
    MessageBoxIcon::Information);

    this->Close();
}
catch (Exception^ e)
{
    MessageBox::Show(
        gcnew System::String(L"Ошибка подключения к базе данных:\n")
+ e->Message,
        gcnew System::String(L"Ошибка"),
        MessageBoxButtons::OK,
        MessageBoxIcon::Error);
}
}

```

Приложение Д. Листинг файла Main.h

```
#pragma once
#include "../include/all_includes.h"
#include "../include/LoginForm.h"
#include "../include/Role.h"
#include "../include/MainFormAdmin.h"
#include "../include/RegisterForm.h"

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow);
```

Приложение Е. Листинг файла Main.cpp

```
#include "../include/Main.h"

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow)
{
    Application::EnableVisualStyles();
    Application::SetCompatibleTextRenderingDefault(false);
    try
    {
        LoginForm^ loginForm = gcnew LoginForm();
        Application::Run(loginForm);

        if (!loginForm->IsConnected)
        {
            return 0;
        }

        RoleSelectionForm^ roleForm = gcnew RoleSelectionForm(loginForm-
>Connection);
        Application::Run(roleForm);

        if (roleForm->SelectedRole == UserRole::None)
        {
            return 0;
        }

        if (roleForm->SelectedRole == UserRole::Admin)
        {
            MainFormAdmin^ adminForm = gcnew MainFormAdmin(loginForm-
>Connection);
            Application::Run(adminForm);
        }
        else if (roleForm->SelectedRole == UserRole::Employee)
        {
            MessageBox::Show(gcnew System::String(L"Форма для сотрудника
будет доступна в следующей версии."));
        }
    }
    catch (Exception^ ex)
    {
        MessageBox::Show(gcnew System::String(L"Критическая ошибка: ") +
ex->Message);
    }
    return 0;
}
```

Приложение Ж. Листинг файла Role.h

```
#pragma once
#include "../include/all_includes.h"

public enum class UserRole
{
    None,
    Admin,
    Employee
};

public ref class RoleSelectionForm : public Form
{
private:
    Button^ btnAdmin;
    Button^ btnEmployee;
    Button^ btnExit;
    GroupBox^ grpLogin;
    TextBox^ txtLogin;
    TextBox^ txtPassword;
    Button^ btnLogin;
    Label^ lblError;

    SqlConnection^ sqlConnection;
    UserRole selectedRole;

    void InitializeComponent();
    bool AuthenticateUser(String^ login, String^ password, int%
employeeId, String^% fullName, String^% role);
    void OnAdminClick(Object^ sender, EventArgs^ args);
    void OnEmployeeClick(Object^ sender, EventArgs^ args);
    void OnLoginClick(Object^ sender, EventArgs^ args);
    void OnExitClick(Object^ sender, EventArgs^ args);

public:
    RoleSelectionForm(SqlConnection^ connection);

    property UserRole SelectedRole
    {
        UserRole get() { return selectedRole; }
    }
};
```

Приложение 3. Листинг файла Role.cpp

```
#include "../include/Role.h"

RoleSelectionForm::RoleSelectionForm(SqlConnection ^ connection)
{
    sqlConnection = connection;
    selectedRole = UserRole::None;
    InitializeComponent();
}

void RoleSelectionForm::InitializeComponent()
{
    this->Text = gcnew System::String(L"Выбор роли пользователя");
    this->Size = System::Drawing::Size(480, 420);
    this->StartPosition = FormStartPosition::CenterScreen;
    this->FormBorderStyle =
System::Windows::Forms::FormBorderStyle::FixedDialog;
    this->MaximizeBox = false;

    Label^ lblTitle = gcnew Label();
    lblTitle->Text = gcnew System::String(L"Система учёта рабочего
времени");
    lblTitle->SetBounds(50, 20, 380, 35);
    lblTitle->Font = gcnew System::Drawing::Font(L"Segoe UI", 14,
FontStyle::Bold);
    lblTitle->TextAlign = ContentAlignment::MiddleCenter;
    this->Controls->Add(lblTitle);

    btnAdmin = gcnew Button();
    btnAdmin->Text = gcnew System::String(L"Администратор");
    btnAdmin->SetBounds(60, 80, 160, 50);
    btnAdmin->Click += gcnew EventHandler(this,
&RoleSelectionForm::OnAdminClick);
    this->Controls->Add(btnAdmin);

    btnEmployee = gcnew Button();
    btnEmployee->Text = gcnew System::String(L"Сотрудник");
    btnEmployee->SetBounds(250, 80, 160, 50);
    btnEmployee->Click += gcnew EventHandler(this,
&RoleSelectionForm::OnEmployeeClick);
    this->Controls->Add(btnEmployee);

    grpLogin = gcnew GroupBox();
    grpLogin->Text = gcnew System::String(L"Вход в систему");
    grpLogin->SetBounds(60, 150, 350, 140);
    grpLogin->Visible = false;
```

```

this->Controls->Add(grpLogin);

Label^ lblLogin = gcnew Label();
lblLogin->Text = gcnew System::String(L"Логин:");
lblLogin->SetBounds(20, 30, 70, 25);
grpLogin->Controls->Add(lblLogin);

txtLogin = gcnew TextBox();
txtLogin->SetBounds(100, 30, 220, 23);
grpLogin->Controls->Add(txtLogin);

Label^ lblPassword = gcnew Label();
lblPassword->Text = gcnew System::String(L"Пароль:");
lblPassword->SetBounds(20, 65, 70, 25);
grpLogin->Controls->Add(lblPassword);

txtPassword = gcnew TextBox();
txtPassword->SetBounds(100, 65, 220, 23);
txtPassword->PasswordChar = '*';
grpLogin->Controls->Add(txtPassword);

btnLogin = gcnew Button();
btnLogin->Text = gcnew System::String(L"Войти");
btnLogin->SetBounds(100, 100, 90, 28);
btnLogin->Click += gcnew EventHandler(this,
&RoleSelectionForm::OnLoginClick);
grpLogin->Controls->Add(btnLogin);

lblError = gcnew Label();
lblError->SetBounds(200, 103, 130, 23);
lblError->ForeColor = Color::Red;
grpLogin->Controls->Add(lblError);

btnExit = gcnew Button();
btnExit->Text = gcnew System::String(L"Выход");
btnExit->SetBounds(370, 330, 80, 30);
btnExit->Click += gcnew EventHandler(this,
&RoleSelectionForm::OnExitClick);
this->Controls->Add(btnExit);
}

bool RoleSelectionForm::AuthenticateUser(
    String^ login,
    String^ password,
    int% employeeId,
    String^% fullName,
    String^% role)

```

```

    {
        if (sqlConnection == nullptr || sqlConnection->State !=
ConnectionState::Open)
        {
            MessageBox::Show(gcnew System::String(L"Нет подключения к базе
данных!"), gcnew System::String(L"Ошибка"));
            return false;
        }

        try
        {
            SqlCommand^ command = gcnew SqlCommand(L"sp_AuthenticateUser",
sqlConnection);

            command->CommandType = CommandType::StoredProcedure;
            command->Parameters->AddWithValue(L"@LoginName", login);
            command->Parameters->AddWithValue(L"@Password", password);

            SqlDataReader^ reader = command->ExecuteReader();

            if (reader->Read())
            {
                employeeId = reader->GetInt32(0);
                fullName = reader->GetString(1) + L" " + reader-
>GetString(2);

                role = reader->GetString(3);
                reader->Close();
                return true;
            }

            reader->Close();
            return false;
        }
        catch (Exception^ ex)
        {
            MessageBox::Show(gcnew System::String(L"Ошибка аутентификации: ")
+ ex->Message);
            return false;
        }
    }

    void RoleSelectionForm::OnAdminClick(Object^ sender, EventArgs^ args)
    {
        selectedRole = UserRole::Admin;
        this->Close();
    }

    void RoleSelectionForm::OnEmployeeClick(Object^ sender, EventArgs^ args)

```



```

{
    grpLogin->Visible = true;
    grpLogin->BringToFront();
}

void RoleSelectionForm::OnLoginClick(Object^ sender, EventArgs^ args)
{
    if (String::IsNullOrEmpty(txtLogin->Text))
    {
        lblError->Text = gcnew System::String(L"Введите логин!");
        txtLogin->Focus();
        return;
    }

    if (String::IsNullOrEmpty(txtPassword->Text))
    {
        lblError->Text = gcnew System::String(L"Введите пароль!");
        txtPassword->Focus();
        return;
    }

    int employeeId;
    String^ fullName;
    String^ role;

    if (AuthenticateUser(txtLogin->Text, txtPassword->Text, employeeId,
fullName, role))
    {
        selectedRole = UserRole::Employee;
        this->Close();
    }
    else
    {
        lblError->Text = gcnew System::String(L"Неверный логин или
пароль!");
        txtPassword->Clear();
        txtPassword->Focus();
    }
}

void RoleSelectionForm::OnExitClick(Object^ sender, EventArgs^ args)
{
    selectedRole = UserRole::None;
    this->Close();
}

```

Приложение И. Листинг файла MainFormAdmin.h

```
#pragma once
#include "../include/all_includes.h"
#include "../include/RegisterForm.h"

public ref class MainFormAdmin : public Form
{
private:
    SqlConnection^ sqlConnection;
    String^ adminPassword;

    TabControl^ tabMain;

    TabPage^ tabEmployees;
    DataGridView^ dgvEmployees;
    Button^ btnAdd;
    Button^ btnEdit;
    Button^ btnDelete;
    Button^ btnRefresh;

    TabPage^ tabTimeEntries;
    ComboBox^ cmbEmployee;
    DateTimePicker^ dtpStartDate;
    DateTimePicker^ dtpEndDate;
    Button^ btnFilter;
    Button^ btnCheckIn;
    Button^ btnCheckOut;
    DataGridView^ dgvTimeEntries;
    Label^ lblLastEntry;

    TabPage^ tabReports;
    DateTimePicker^ dtpReportStart;
    DateTimePicker^ dtpReportEnd;
    Button^ btnGenerateReport;
    Button^ btnExportCsv;
    DataGridView^ dgvReport;
    SaveFileDialog^ saveFileDialog;

    void InitializeComponent();
    void LoadEmployees();
    void LoadEmployeeCombo();
    void LoadTimeEntries();
    void UpdateLastEntryInfo();
    void LoadReport();
    void ExportToCsv(DataTable^ dataTable, String^ filePath);
    bool VerifyAdminPassword();
```

```
void OnAddClick(Object^ sender, EventArgs^ args);  
void OnEditClick(Object^ sender, EventArgs^ args);  
void OnDeleteClick(Object^ sender, EventArgs^ args);  
void OnRefreshClick(Object^ sender, EventArgs^ args);  
void OnEmployeesSelectionChanged(Object^ sender, EventArgs^ args);  
void OnFilterClick(Object^ sender, EventArgs^ args);  
void OnCheckInClick(Object^ sender, EventArgs^ args);  
void OnCheckOutClick(Object^ sender, EventArgs^ args);  
void OnGenerateReportClick(Object^ sender, EventArgs^ args);  
void OnExportCsvClick(Object^ sender, EventArgs^ args);  
  
public:  
    MainFormAdmin(SqlConnection^ connection);  
};
```

Приложение К. Листинг файла MainFormAdmin.cpp

```
#include "../include/MainFormAdmin.h"

MainFormAdmin::MainFormAdmin(SqlConnection ^ connection)
{
    sqlConnection = connection;
    adminPassword = L"admin123";
    InitializeComponent();
    LoadEmployees();
    LoadEmployeeCombo();
    LoadTimeEntries();
}

void MainFormAdmin::InitializeComponent()
{
    this->Text = gcnew System::String(L"Панель администратора");
    this->Size = System::Drawing::Size(1200, 750);
    this->StartPosition = FormStartPosition::CenterScreen;
    this->WindowState = FormWindowState::Maximized;

    tabMain = gcnew TabControl();
    tabMain->Dock = DockStyle::Fill;
    this->Controls->Add(tabMain);

    tabEmployees = gcnew TabPage(gcnew System::String(L"Управление
сотрудниками"));
    tabMain->TabPage->Add(tabEmployees);

    Panel^ panelButtons = gcnew Panel();
    panelButtons->SetBounds(10, 10, 1150, 45);
    tabEmployees->Controls->Add(panelButtons);

    btnAdd = gcnew Button();
    btnAdd->Text = gcnew System::String(L"Добавить сотрудника");
    btnAdd->SetBounds(10, 8, 150, 30);
    btnAdd->Click += gcnew EventHandler(this,
&MainFormAdmin::OnAddClick);
    panelButtons->Controls->Add(btnAdd);

    btnEdit = gcnew Button();
    btnEdit->Text = gcnew System::String(L"Редактировать");
    btnEdit->SetBounds(170, 8, 120, 30);
    btnEdit->Enabled = false;
    btnEdit->Click += gcnew EventHandler(this,
&MainFormAdmin::OnEditClick);
    panelButtons->Controls->Add(btnEdit);
```

```

btnDelete = gcnew Button();
btnDelete->Text = gcnew System::String(L"Удалить");
btnDelete->SetBounds(300, 8, 120, 30);
btnDelete->Enabled = false;
btnDelete->Click += gcnew EventHandler(this,
&MainFormAdmin::OnDeleteClick);
panelButtons->Controls->Add(btnDelete);

btnRefresh = gcnew Button();
btnRefresh->Text = gcnew System::String(L"Обновить");
btnRefresh->SetBounds(430, 8, 120, 30);
btnRefresh->Click += gcnew EventHandler(this,
&MainFormAdmin::OnRefreshClick);
panelButtons->Controls->Add(btnRefresh);

dgvEmployees = gcnew DataGridView();
dgvEmployees->SetBounds(10, 60, 1150, 600);
dgvEmployees->ReadOnly = true;
dgvEmployees->SelectionMode =
DataGridViewSelectionMode::FullRowSelect;
dgvEmployees->MultiSelect = false;
dgvEmployees->AllowUserToAddRows = false;
dgvEmployees->AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode::Fill;
dgvEmployees->SelectionChanged += gcnew EventHandler(this,
&MainFormAdmin::OnEmployeesSelectionChanged);
tabEmployees->Controls->Add(dgvEmployees);

tabTimeEntries = gcnew TabPage(gcnew System::String(L"Отметки
времени"));
tabMain->TabPagees->Add(tabTimeEntries);

Panel^ panelFilter = gcnew Panel();
panelFilter->SetBounds(10, 10, 1150, 50);
tabTimeEntries->Controls->Add(panelFilter);

Label^ lblEmployee = gcnew Label();
lblEmployee->Text = gcnew System::String(L"Сотрудник:");
lblEmployee->SetBounds(10, 15, 80, 25);
panelFilter->Controls->Add(lblEmployee);

cmbEmployee = gcnew ComboBox();
cmbEmployee->SetBounds(90, 13, 200, 25);
cmbEmployee->DropDownStyle = ComboBoxStyle::DropDownList;
panelFilter->Controls->Add(cmbEmployee);

```

```

Label^ lblStart = gcnew Label();
lblStart->Text = gcnew System::String(L"С даты:");
lblStart->SetBounds(310, 15, 60, 25);
panelFilter->Controls->Add(lblStart);

dtpStartDate = gcnew DateTimePicker();
dtpStartDate->SetBounds(370, 13, 120, 25);
dtpStartDate->Format = DateTimePickerFormat::Short;
dtpStartDate->Value = DateTime::Now.AddDays(-30);
panelFilter->Controls->Add(dtpStartDate);

Label^ lblEnd = gcnew Label();
lblEnd->Text = gcnew System::String(L"По дате:");
lblEnd->SetBounds(510, 15, 60, 25);
panelFilter->Controls->Add(lblEnd);

dtpEndDate = gcnew DateTimePicker();
dtpEndDate->SetBounds(570, 13, 120, 25);
dtpEndDate->Format = DateTimePickerFormat::Short;
dtpEndDate->Value = DateTime::Now;
panelFilter->Controls->Add(dtpEndDate);

btnFilter = gcnew Button();
btnFilter->Text = gcnew System::String(L"Показать");
btnFilter->SetBounds(710, 11, 100, 28);
btnFilter->Click += gcnew EventHandler(this,
&MainFormAdmin::OnFilterClick);
panelFilter->Controls->Add(btnFilter);

btnCheckIn = gcnew Button();
btnCheckIn->Text = gcnew System::String(L"Отметить приход");
btnCheckIn->SetBounds(830, 11, 130, 28);
btnCheckIn->Click += gcnew EventHandler(this,
&MainFormAdmin::OnCheckInClick);
panelFilter->Controls->Add(btnCheckIn);

btnCheckOut = gcnew Button();
btnCheckOut->Text = gcnew System::String(L"Отметить уход");
btnCheckOut->SetBounds(970, 11, 130, 28);
btnCheckOut->Click += gcnew EventHandler(this,
&MainFormAdmin::OnCheckOutClick);
panelFilter->Controls->Add(btnCheckOut);

lblLastEntry = gcnew Label();
lblLastEntry->Text = gcnew System::String(L"Последняя отметка: ---");
lblLastEntry->SetBounds(10, 65, 500, 25);
tabTimeEntries->Controls->Add(lblLastEntry);

```

```

dgvTimeEntries = gcnew DataGridView();
dgvTimeEntries->SetBounds(10, 95, 1150, 565);
dgvTimeEntries->ReadOnly = true;
dgvTimeEntries->AllowUserToAddRows = false;
dgvTimeEntries->AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode::Fill;
tabTimeEntries->Controls->Add(dgvTimeEntries);

tabReports = gcnew TabPage(gcnew System::String(L"Формирование
отчётов"));
tabMain->TabPage->Add(tabReports);

Panel^ panelReport = gcnew Panel();
panelReport->SetBounds(10, 10, 1150, 50);
tabReports->Controls->Add(panelReport);

Label^ lblReportStart = gcnew Label();
lblReportStart->Text = gcnew System::String(L"Начало периода:");
lblReportStart->SetBounds(10, 15, 100, 25);
panelReport->Controls->Add(lblReportStart);

dtpReportStart = gcnew DateTimePicker();
dtpReportStart->SetBounds(115, 13, 120, 25);
dtpReportStart->Format = DateTimePickerFormat::Short;
dtpReportStart->Value = DateTime(DateTime::Now.Year,
DateTime::Now.Month, 1);
panelReport->Controls->Add(dtpReportStart);

Label^ lblReportEnd = gcnew Label();
lblReportEnd->Text = gcnew System::String(L"Конец периода:");
lblReportEnd->SetBounds(250, 15, 100, 25);
panelReport->Controls->Add(lblReportEnd);

dtpReportEnd = gcnew DateTimePicker();
dtpReportEnd->SetBounds(355, 13, 120, 25);
dtpReportEnd->Format = DateTimePickerFormat::Short;
dtpReportEnd->Value = DateTime::Now;
panelReport->Controls->Add(dtpReportEnd);

btnGenerateReport = gcnew Button();
btnGenerateReport->Text = gcnew System::String(L"Сформировать
отчёт");
btnGenerateReport->SetBounds(500, 11, 150, 28);
btnGenerateReport->Click += gcnew EventHandler(this,
&MainFormAdmin::OnGenerateReportClick);

```

```

panelReport->Controls->Add(btnGenerateReport);

btnExportCsv = gcnew Button();
btnExportCsv->Text = gcnew System::String(L"Экспорт в CSV");
btnExportCsv->SetBounds(660, 11, 120, 28);
btnExportCsv->Click += gcnew EventHandler(this,
&MainFormAdmin::OnExportCsvClick);
panelReport->Controls->Add(btnExportCsv);

dgvReport = gcnew DataGridView();
dgvReport->SetBounds(10, 65, 1150, 595);
dgvReport->ReadOnly = true;
dgvReport->AllowUserToAddRows = false;
dgvReport->AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode::Fill;
tabReports->Controls->Add(dgvReport);

saveFileDialog = gcnew SaveFileDialog();
saveFileDialog->Filter = L"CSV файлы (*.csv)|*.csv";
saveFileDialog->DefaultExt = L"csv";
}

void MainFormAdmin::LoadEmployees()
{
    try
    {
        SqlCommand^ command = gcnew SqlCommand(L"sp_GetAllEmployees",
sqlConnection);

        command->CommandType = CommandType::StoredProcedure;

        SqlDataAdapter^ adapter = gcnew SqlDataAdapter(command);
        DataTable^ dataTable = gcnew DataTable();
        adapter->Fill(dataTable);

        dgvEmployees->DataSource = dataTable;

        if (dgvEmployees->Columns->Count > 0)
        {
            dgvEmployees->Columns["Employee_ID"]->HeaderText = L"ID";
            dgvEmployees->Columns["Last_Name"]->HeaderText = L"Фамилия";
            dgvEmployees->Columns["First_Name"]->HeaderText = L"Имя";
            dgvEmployees->Columns["Middle_Name"]->HeaderText =
L"Отчество";

            dgvEmployees->Columns["Position"]->HeaderText = L"Должность";
            dgvEmployees->Columns["Login_Name"]->HeaderText = L"Логин";
            dgvEmployees->Columns["Role_Name"]->HeaderText = L"Роль";
        }
    }
}

```



```

        dgvEmployees->Columns["Hire_Date"]->HeaderText = L"Дата
приёма";

        dgvEmployees->Columns["Is_Active"]->HeaderText = L"Активен";
    }
}
catch (Exception^ ex)
{
    MessageBox::Show(gcnew System::String(L"Ошибка загрузки
сотрудников: ") + ex->Message);
}

}

void MainFormAdmin::LoadEmployeeCombo()
{
    try
    {
        SqlCommand^ command = gcnew SqlCommand(
            L"SELECT Employee_ID, Last_Name + ' ' + First_Name AS
FullName FROM Employee WHERE Is_Active = 1",
            sqlConnection);

        SqlDataReader^ reader = command->ExecuteReader();
        cmbEmployee->Items->Clear();

        while (reader->Read())
        {
            String^ item = reader["Employee_ID"]->ToString() + L" - " +
reader["FullName"]->ToString();
            cmbEmployee->Items->Add(item);
        }
        reader->Close();

        if (cmbEmployee->Items->Count > 0) cmbEmployee->SelectedIndex =
0;
    }
    catch (Exception^ ex)
    {
        MessageBox::Show(gcnew System::String(L"Ошибка загрузки списка:
") + ex->Message);
    }
}

void MainFormAdmin::LoadTimeEntries()
{
    if (cmbEmployee->SelectedIndex == -1) return;

    try

```

```

{
    String^ selectedItem = cmbEmployee->SelectedItem->ToString();
    int employeeId = Int32::Parse(selectedItem->Substring(0,
selectedItem->IndexOf(L" -")));

    SqlCommand^ command = gcnew
SqlCommand(L"sp_GetTimeEntriesForPeriod", sqlConnection);
    command->CommandType = CommandType::StoredProcedure;
    command->Parameters->AddWithValue(L"@EmployeeID", employeeId);
    command->Parameters->AddWithValue(L"@StartDate", dtpStartDate-
>Value.ToString(L"yyyy-MM-dd"));
    command->Parameters->AddWithValue(L"@EndDate", dtpEndDate-
>Value.ToString(L"yyyy-MM-dd"));

    SqlDataAdapter^ adapter = gcnew SqlDataAdapter(command);
    DataTable^ dataTable = gcnew DataTable();
    adapter->Fill(dataTable);

    dgvTimeEntries->DataSource = dataTable;

    if (dgvTimeEntries->Columns->Count > 0)
    {
        dgvTimeEntries->Columns["TimeEntry_ID"]->Visible = false;
        dgvTimeEntries->Columns["Entry_Type"]->HeaderText = L"Тип";
        dgvTimeEntries->Columns["EntryDate"]->HeaderText = L"Дата";
        dgvTimeEntries->Columns["EntryTimeOnly"]->HeaderText =
L"Время";

        dgvTimeEntries->Columns["Entry_Time"]->Visible = false;
    }

    UpdateLastEntryInfo();
}
catch (Exception^ ex)
{
    MessageBox::Show(gcnew System::String(L"Ошибка загрузки отметок:
") + ex->Message);
}

void MainFormAdmin::UpdateLastEntryInfo()
{
    if (cmbEmployee->SelectedIndex == -1) return;

    try
    {
        String^ selectedItem = cmbEmployee->SelectedItem->ToString();

```

```

        int employeeId = Int32::Parse(selectedItem->Substring(0,
selectedItem->IndexOf(L" -")));

        SqlCommand^ command = gcnew SqlCommand(L"sp_GetLastTimeEntry",
sqlConnection);
        command->CommandType = CommandType::StoredProcedure;
        command->Parameters->AddWithValue(L"@EmployeeID", employeeId);

        SqlDataReader^ reader = command->ExecuteReader();

        if (reader->Read())
        {
            String^ entryType = reader["Entry_Type"]->ToString();
            String^ formattedTime = reader["FormattedTime"]->ToString();
            String^ typeText = (entryType == L"CheckIn") ? L"ПРИХОД" :
L"УХОД";

            lblLastEntry->Text = gcnew System::String(L"Последняя
отметка: ") + typeText + L" - " + formattedTime;
            lblLastEntry->ForeColor = (entryType == L"CheckIn") ?
Color::Green : Color::Red;
        }
        else
        {
            lblLastEntry->Text = gcnew System::String(L"Последняя
отметка: нет данных");
            lblLastEntry->ForeColor = Color::Gray;
        }
        reader->Close();
    }
    catch (Exception^)
    {
        lblLastEntry->Text = gcnew System::String(L"Последняя отметка:
ошибка");
    }
}

void MainFormAdmin::LoadReport()
{
    try
    {
        SqlCommand^ command = gcnew SqlCommand(L"sp_GetTimeReport",
sqlConnection);
        command->CommandType = CommandType::StoredProcedure;
        command->Parameters->AddWithValue(L"@StartDate", dtpReportStart-
>Value.ToString(L"yyyy-MM-dd"));
        command->Parameters->AddWithValue(L"@EndDate", dtpReportEnd-
>Value.ToString(L"yyyy-MM-dd"));
    }
}

```

```

SqlDataAdapter^ adapter = gcnew SqlDataAdapter(command);
DataTable^ dataTable = gcnew DataTable();
adapter->Fill(dataTable);

dgvReport->DataSource = dataTable;

if (dgvReport->Columns->Count > 0)
{
    dgvReport->Columns["Employee_ID"]->Visible = false;
    dgvReport->Columns["FullName"]->HeaderText = L"ФИО
сотрудника";

    dgvReport->Columns["Position"]->HeaderText = L"Должность";
    dgvReport->Columns["WorkDate"]->HeaderText = L"Дата";
    dgvReport->Columns["CheckInTime"]->HeaderText = L"Время
прихода";

    dgvReport->Columns["CheckOutTime"]->HeaderText = L"Время
ухода";

    dgvReport->Columns["WorkDate"]->DefaultCellStyle->Format =
L"dd.MM.yyyy";
    dgvReport->Columns["CheckInTime"]->DefaultCellStyle->Format =
L"dd.MM.yyyy HH:mm:ss";
    dgvReport->Columns["CheckOutTime"]->DefaultCellStyle->Format
= L"dd.MM.yyyy HH:mm:ss";
}
}
catch (Exception^ ex)
{
    MessageBox::Show(gcnew System::String(L"Ошибка формирования
отчёта: ") + ex->Message);
}
}

void MainFormAdmin::ExportToCsv(DataTable^ dataTable, String^ filePath)
{
    StreamWriter^ writer = gcnew StreamWriter(filePath, false,
System::Text::Encoding::UTF8);
    try
    {
        writer->WriteLine(L"ФИО сотрудника;Должность;Дата;Время
прихода;Время ухода");

        for each (DataRow ^ row in dataTable->Rows)
        {

```

```

String^ fullName = row["FullName"]->ToString();
String^ position = row["Position"]->ToString();
String^ workDate = row["WorkDate"]->ToString();

String^ checkInTime = (row["CheckInTime"] != DBNull::Value) ?
row["CheckInTime"]->ToString() : L"";
String^ checkOutTime = (row["CheckOutTime"] != DBNull::Value)
? row["CheckOutTime"]->ToString() : L"";

writer->WriteLine(String::Format(L"{0};{1};{2};{3};{4}",
    fullName, position, workDate, checkInTime,
checkOutTime));
}

MessageBox::Show(gcnew System::String(L"Отчёт сохранён!\n\nПапка:
") +
    Path::GetDirectoryName(filePath) + L"\nфайл: " +
Path::GetFileName(filePath),
    gcnew System::String(L"Успех"), MessageBoxButtons::OK,
MessageBoxIcon::Information);
}
finally
{
    writer->Close();
}
}

bool MainFormAdmin::VerifyAdminPassword()
{
    Form^ passwordForm = gcnew Form();
    passwordForm->Text = gcnew System::String(L"Подтверждение пароля");
    passwordForm->Size = System::Drawing::Size(380, 170);
    passwordForm->StartPosition = FormStartPosition::CenterParent;
    passwordForm->FormBorderStyle =
System::Windows::Forms::FormBorderStyle::FixedDialog;

    Label^ lblMessage = gcnew Label();
    lblMessage->Text = gcnew System::String(L"Введите пароль
администратора:");
    lblMessage->SetBounds(20, 20, 250, 25);
    passwordForm->Controls->Add(lblMessage);

    TextBox^ txtPassword = gcnew TextBox();
    txtPassword->SetBounds(20, 50, 320, 23);
    txtPassword->PasswordChar = '*';
    passwordForm->Controls->Add(txtPassword);

```

```

        Button^ btnOk = gnew Button();
        btnOk->Text = gnew System::String(L"Подтвердить");
        btnOk->SetBounds(130, 90, 100, 30);
        btnOk->DialogResult = System::Windows::Forms::DialogResult::OK;
        passwordForm->Controls->Add(btnOk);

        if (passwordForm->ShowDialog() ==
System::Windows::Forms::DialogResult::OK)
        {
            return txtPassword->Text == adminPassword;
        }
        return false;
    }

    void MainFormAdmin::OnAddClick(Object^ sender, EventArgs^ args)
    {
        if (!VerifyAdminPassword())
        {
            MessageBox::Show(gnew System::String(L"Неверный пароль!"), gnew
System::String(L"Доступ запрещён"));
            return;
        }
        RegisterForm^ regForm = gnew RegisterForm(sqlConnection);
        regForm->ShowDialog();
        LoadEmployees();
        LoadEmployeeCombo();
    }

    void MainFormAdmin::OnEditClick(Object^ sender, EventArgs^ args)
    {
        if (dgvEmployees->SelectedRows->Count == 0) return;
        if (!VerifyAdminPassword())
        {
            MessageBox::Show(gnew System::String(L"Неверный пароль!"), gnew
System::String(L"Доступ запрещён"));
            return;
        }
        int employeeId = Convert::ToInt32(dgvEmployees->SelectedRows[0]-
>Cells["Employee_ID"]->Value);
        RegisterForm^ editForm = gnew RegisterForm(sqlConnection,
employeeId);
        editForm->ShowDialog();
        LoadEmployees();
        LoadEmployeeCombo();
    }

    void MainFormAdmin::onDeleteClick(Object^ sender, EventArgs^ args)

```

```

{
    if (dgvEmployees->SelectedRows->Count == 0) return;
    if (!VerifyAdminPassword())
    {
        MessageBox::Show(gcnew System::String(L"Неверный пароль!"), gcnew
System::String(L"Доступ запрещён"));
        return;
    }

    System::Windows::Forms::DialogResult result = MessageBox::Show(
        gcnew System::String(L"Удалить сотрудника? Все отметки времени
будут удалены!"),
        gcnew System::String(L"Подтверждение"), MessageBoxButtons::YesNo,
MessageBoxIcon::Warning);

    if (result == System::Windows::Forms::DialogResult::Yes)
    {
        try
        {
            int employeeId = Convert::ToInt32(dgvEmployees-
>SelectedRows[0]->Cells["Employee_ID"]->Value);
            SqlCommand^ command = gcnew SqlCommand(L"sp_DeleteEmployee",
sqlConnection);

            command->CommandType = CommandType::StoredProcedure;
            command->Parameters->AddWithValue(L"@EmployeeID",
employeeId);

            command->ExecuteNonQuery();
            MessageBox::Show(gcnew System::String(L"Сотрудник удалён!"),
gcnew System::String(L"Успех"));
            LoadEmployees();
            LoadEmployeeCombo();
            LoadTimeEntries();
        }
        catch (Exception^ ex) { MessageBox::Show(gcnew
System::String(L"Ошибка: ") + ex->Message); }
    }
}

void MainFormAdmin::OnRefreshClick(Object^ sender, EventArgs^ args)
{
    LoadEmployees();
    LoadEmployeeCombo();
}

void MainFormAdmin::OnEmployeesSelectionChanged(Object^ sender,
EventArgs^ args)
{

```

```

        bool hasSelection = dgvEmployees->SelectedRows->Count > 0;
        btnEdit->Enabled = hasSelection;
        btnDelete->Enabled = hasSelection;
    }

    void MainFormAdmin::OnFilterClick(Object^ sender, EventArgs^ args)
    {
        LoadTimeEntries();
    }

    void MainFormAdmin::OnCheckInClick(Object^ sender, EventArgs^ args)
    {
        if (cmbEmployee->SelectedIndex == -1)
        {
            MessageBox::Show(gcnew System::String(L"Выберите сотрудника!"));
            return;
        }

        String^ selectedItem = cmbEmployee->SelectedItem->ToString();
        int employeeId = Int32::Parse(selectedItem->Substring(0,
selectedItem->IndexOf(L" -")));

        try
        {
            SqlCommand^ command = gcnew SqlCommand(L"sp_AddTimeEntry",
sqlConnection);

            command->CommandType = CommandType::StoredProcedure;
            command->Parameters->AddWithValue(L"@EmployeeID", employeeId);
            command->Parameters->AddWithValue(L"@EntryType", L"CheckIn");
            command->ExecuteNonQuery();

            MessageBox::Show(gcnew System::String(L"Отметка о приходе
добавлена!"));

            LoadTimeEntries();
        }
        catch (Exception^ ex) { MessageBox::Show(gcnew
System::String(L"Ошибка: ") + ex->Message); }
    }

    void MainFormAdmin::OnCheckOutClick(Object^ sender, EventArgs^ args)
    {
        if (cmbEmployee->SelectedIndex == -1)
        {
            MessageBox::Show(gcnew System::String(L"Выберите сотрудника!"));
            return;
        }

        String^ selectedItem = cmbEmployee->SelectedItem->ToString();

```



```

        int employeeId = Int32::Parse(selectedItem->Substring(0,
selectedItem->IndexOf(L" -")));

        try
        {
            SqlCommand^ command = gcnew SqlCommand(L"sp_AddTimeEntry",
sqlConnection);

            command->CommandType = CommandType::StoredProcedure;
            command->Parameters->AddWithValue(L"@EmployeeID", employeeId);
            command->Parameters->AddWithValue(L"@EntryType", L"CheckOut");
            command->ExecuteNonQuery();

            MessageBox::Show(gcnew System::String(L"Отметка об уходе
добавлена!"));

            LoadTimeEntries();
        }
        catch (Exception^ ex) { MessageBox::Show(gcnew
System::String(L"Ошибка: ") + ex->Message); }
    }

    void MainFormAdmin::OnGenerateReportClick(Object^ sender, EventArgs^
args)
    {
        LoadReport();
    }

    void MainFormAdmin::OnExportCsvClick(Object^ sender, EventArgs^ args)
    {
        if (dgvReport->Rows->Count == 0)
        {
            MessageBox::Show(gcnew System::String(L"Нет данных для экспорта!
Сначала сформируйте отчёт."),
                gcnew System::String(L"Ошибка"), MessageBoxButtons::OK,
MessageBoxIcon::Warning);

            return;
        }

        try
        {
            // Получаем путь к папке, где находится EXE-файл
            String^ exePath = Assembly::GetExecutingAssembly()->Location;
            String^ exeDirectory = Path::GetDirectoryName(exePath);
            String^ reportsPath = Path::Combine(exeDirectory, L"Reports");

            // Для отладки: показываем путь в сообщении
            MessageBox::Show(gcnew System::String(L"Путь для сохранения:\n")
+ reportsPath,

```

```

        gcnew System::String(L"Информация"), MessageBoxButtons::OK,
        MessageBoxIcon::Information);

        // Создаём папку Reports, если её нет
        if (!Directory::Exists(reportsPath))
        {
            Directory::CreateDirectory(reportsPath);
            MessageBox::Show(gcnew System::String(L"Папка Reports
создана!"),
                gcnew System::String(L"Информация"),
        MessageBoxButtons::OK, MessageBoxIcon::Information);
        }

        // Формируем имя файла
        String^ fileName = L"Report_" + DateTime::Now.ToString(L"yyyy-MM-
dd_HH-mm-ss") + L".csv";
        String^ filePath = Path::Combine(reportsPath, fileName);

        // Получаем данные
        DataTable^ dataTable = safe_cast<DataTable^>(dgvReport-
>DataSource);

        // Экспортируем
        ExportToCsv(dataTable, filePath);
    }
    catch (Exception^ ex)
    {
        MessageBox::Show(gcnew System::String(L"Ошибка экспорта: ") + ex-
>Message,
            gcnew System::String(L"Ошибка"), MessageBoxButtons::OK,
        MessageBoxIcon::Error);
    }
}

```

Приложение Л. Листинг файла RegisterForm.h

```
#pragma once
#include "../include/all_includes.h"

public ref class RegisterForm : public Form
{
private:
    SqlConnection^ sqlConnection;
    bool isEditMode;
    int editingEmployeeId;
    bool isRegistered;
    int newEmployeeId;

    TextBox^ txtLastName;
    TextBox^ txtFirstName;
    TextBox^ txtMiddleName;
    ComboBox^ cmbPosition;
    TextBox^ txtLogin;
    TextBox^ txtPassword;
    TextBox^ txtConfirmPassword;
    ComboBox^ cmbRole;
    CheckBox^ chkIsActive;

    Button^ btnSave;
    Button^ btnCancel;

    void InitializeComponent();
    void LoadPositions();
    void LoadEmployeeData();
    bool ValidateInputs();
    void OnSaveClick(Object^ sender, EventArgs^ args);
    void OnCancelClick(Object^ sender, EventArgs^ args);

public:
    RegisterForm(SqlConnection^ connection);
    RegisterForm(SqlConnection^ connection, int employeeId);

    property bool IsRegistered { bool get() { return isRegistered; } }
    property int NewEmployeeId { int get() { return newEmployeeId; } }
};
```

Приложение М. Листинг файла RegisterForm.cpp

```
#include "../include/RegisterForm.h"

RegisterForm::RegisterForm(SqlConnection ^ connection)
{
    sqlConnection = connection;
    isEditMode = false;
    editingEmployeeId = -1;
    isRegistered = false;
    newEmployeeId = -1;
    InitializeComponent();
    LoadPositions();
}

RegisterForm::RegisterForm(SqlConnection^ connection, int employeeId)
{
    sqlConnection = connection;
    isEditMode = true;
    editingEmployeeId = employeeId;
    isRegistered = false;
    newEmployeeId = -1;
    InitializeComponent();
    LoadPositions();
    LoadEmployeeData();
}

void RegisterForm::InitializeComponent()
{
    if (isEditMode) this->Text = gcnew System::String(L"Редактирование
сотрудника");
    else this->Text = gcnew System::String(L"Регистрация сотрудника");

    this->Size = System::Drawing::Size(480, 550);
    this->StartPosition = FormStartPosition::CenterParent;
    this->FormBorderStyle =
System::Windows::Forms::FormBorderStyle::FixedDialog;
    this->MaximizeBox = false;

    int yPos = 20, leftMargin = 20, labelWidth = 110, fieldWidth = 300;

    Label^ lblLastName = gcnew Label(); lblLastName->Text = gcnew
System::String(L"Фамилия:*"); lblLastName->SetBounds(leftMargin, yPos,
labelWidth, 25); this->Controls->Add(lblLastName);
    txtLastName = gcnew TextBox(); txtLastName->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 25); this->Controls->Add(txtLastName); yPos +=
35;
```

```

        Label^ lblFirstName = gcnew Label(); lblFirstName->Text = gcnew
System::String(L"Имя:"); lblFirstName->SetBounds(leftMargin, yPos, labelWidth,
25); this->Controls->Add(lblFirstName);

        txtFirstName = gcnew TextBox(); txtFirstName->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 25); this->Controls->Add(txtFirstName); yPos +=
35;

        Label^ lblMiddleName = gcnew Label(); lblMiddleName->Text = gcnew
System::String(L"Отчество:"); lblMiddleName->SetBounds(leftMargin, yPos,
labelWidth, 25); this->Controls->Add(lblMiddleName);

        txtMiddleName = gcnew TextBox(); txtMiddleName->SetBounds(leftMargin
+ labelWidth, yPos, fieldWidth, 25); this->Controls->Add(txtMiddleName); yPos
+= 35;

        Label^ lblPosition = gcnew Label(); lblPosition->Text = gcnew
System::String(L"Должность:"); lblPosition->SetBounds(leftMargin, yPos,
labelWidth, 25); this->Controls->Add(lblPosition);

        cmbPosition = gcnew ComboBox(); cmbPosition->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 27); cmbPosition->DropDownStyle =
ComboBoxStyle::DropDownList; this->Controls->Add(cmbPosition); yPos += 35;

        Label^ lblLogin = gcnew Label(); lblLogin->Text = gcnew
System::String(L"Логин:"); lblLogin->SetBounds(leftMargin, yPos, labelWidth,
25); this->Controls->Add(lblLogin);

        txtLogin = gcnew TextBox(); txtLogin->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 25); this->Controls->Add(txtLogin); yPos += 35;

        Label^ lblPassword = gcnew Label(); lblPassword->Text = gcnew
System::String(L"Пароль:"); lblPassword->SetBounds(leftMargin, yPos,
labelWidth, 25); this->Controls->Add(lblPassword);

        txtPassword = gcnew TextBox(); txtPassword->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 25); txtPassword->PasswordChar = '*'; this-
>Controls->Add(txtPassword); yPos += 35;

        Label^ lblConfirm = gcnew Label(); lblConfirm->Text = gcnew
System::String(L"Подтверждение:"); lblConfirm->SetBounds(leftMargin, yPos,
labelWidth, 25); this->Controls->Add(lblConfirm);

        txtConfirmPassword = gcnew TextBox(); txtConfirmPassword-
>SetBounds(leftMargin + labelWidth, yPos, fieldWidth, 25); txtConfirmPassword-
>PasswordChar = '*'; this->Controls->Add(txtConfirmPassword); yPos += 35;

        Label^ lblRole = gcnew Label(); lblRole->Text = gcnew
System::String(L"Роль:"); lblRole->SetBounds(leftMargin, yPos, labelWidth,
25); this->Controls->Add(lblRole);

```

```

        cmbRole = gcnw ComboBox(); cmbRole->SetBounds(leftMargin +
labelWidth, yPos, fieldWidth, 27); cmbRole->DropDownStyle =
ComboBoxStyle::DropDownList;
        cmbRole->Items->Add(L"Employee"); cmbRole->Items->Add(L"Admin");
cmbRole->SelectedIndex = 0; this->Controls->Add(cmbRole); yPos += 35;

        if (isEditMode)
        {
            chkIsActive = gcnw CheckBox(); chkIsActive->Text = gcnw
System::String(L"Сотрудник активен");
            chkIsActive->SetBounds(leftMargin + labelWidth, yPos, 150, 25);
chkIsActive->Checked = true; this->Controls->Add(chkIsActive); yPos += 40;
        }

        btnSave = gcnw Button(); btnSave->Text = isEditMode ? gcnw
System::String(L"Сохранить") : gcnw System::String(L"Зарегистрировать");
        btnSave->SetBounds(leftMargin + 60, yPos, 120, 35); btnSave->Click +=
gcnw EventHandler(this, &RegisterForm::OnSaveClick); this->Controls-
>Add(btnSave);

        btnCancel = gcnw Button(); btnCancel->Text = gcnw
System::String(L"Отмена");
        btnCancel->SetBounds(leftMargin + 200, yPos, 100, 35); btnCancel-
>Click += gcnw EventHandler(this, &RegisterForm::OnCancelClick); this-
>Controls->Add(btnCancel); yPos += 45;

        Label^ lblRequired = gcnw Label(); lblRequired->Text = gcnw
System::String(L"* Поля, обязательные для заполнения");
        lblRequired->SetBounds(leftMargin, yPos, 250, 25); lblRequired-
>ForeColor = Color::Red; this->Controls->Add(lblRequired);
    }

    void RegisterForm::LoadPositions()
    {
        try
        {
            SqlCommand^ command = gcnw SqlCommand(L"sp_GetAllPositions",
sqlConnection);
            command->CommandType = CommandType::StoredProcedure;

            SqlDataReader^ reader = command->ExecuteReader();
            cmbPosition->Items->Clear();
            while (reader->Read()) cmbPosition->Items-
>Add(reader["Position_Name"]->ToString());
            reader->Close();
            if (cmbPosition->Items->Count > 0) cmbPosition->SelectedIndex =
0;

```

```

    }

    catch (Exception^ ex) { MessageBox::Show(gcnew
System::String(L"Ошибка загрузки должностей: ") + ex->Message); }

    }

    void RegisterForm::LoadEmployeeData()
    {
        try
        {
            SqlCommand^ command = gcnew SqlCommand(
                L"SELECT e.Last_Name, e.First_Name, e.Middle_Name,
p.Position_Name, "
                L"e.Login_Name, e.Role_Name, e.Is_Active "
                L"FROM Employee e INNER JOIN Position p ON e.Position_ID =
p.Position_ID "
                L"WHERE e.Employee_ID = @EmployeeID", sqlConnection);
            command->Parameters->AddWithValue(L"@EmployeeID",
editingEmployeeId);

            SqlDataReader^ reader = command->ExecuteReader();
            if (reader->Read())
            {
                txtLastName->Text = reader["Last_Name"]->ToString();
                txtFirstName->Text = reader["First_Name"]->ToString();
                txtMiddleName->Text = (reader["Middle_Name"] !=
DBNull::Value) ? reader["Middle_Name"]->ToString() : L"";
                cmbPosition->SelectedItem = reader["Position_Name"]->
ToString();

                txtLogin->Text = reader["Login_Name"]->ToString();
                cmbRole->SelectedItem = reader["Role_Name"]->ToString();
                if (isEditMode) chkIsActive->Checked =
Convert::ToBoolean(reader["Is_Active"]);

                txtPassword->Text = L"*****";
                txtConfirmPassword->Text = L"*****";
                txtPassword->Enabled = false;
                txtConfirmPassword->Enabled = false;
            }
            reader->Close();
        }
        catch (Exception^ ex) { MessageBox::Show(gcnew
System::String(L"Ошибка загрузки: ") + ex->Message); }
    }

    bool RegisterForm::ValidateInputs()
    {
        // Проверка фамилии

```

```

Regex^ lastNameRegex = gcnew Regex(L"^[А-ЯЁ][а-яё]*(?:-[А-ЯЁ][а-
яё]*)?$");
if (!lastNameRegex->IsMatch(txtLastName->Text))
{
    MessageBox::Show(L"Фамилия должна начинаться с заглавной буквы,
содержать только русские буквы. Допускается дефис (например: Иванов, Убей-
волк)");
    txtLastName->Focus();
    return false;
}

// Проверка имени
Regex^ firstNameRegex = gcnew Regex(L"^[А-ЯЁ][а-яё]*(?:-[А-ЯЁ][а-
яё]*)?$");
if (!firstNameRegex->IsMatch(txtFirstName->Text))
{
    MessageBox::Show(L"Имя должно начинаться с заглавной буквы и
содержать только русские буквы. Допускается дефис (например: Анна-Мария)");
    txtFirstName->Focus();
    return false;
}

// Проверка отчества (необязательное поле)
if (!String::IsNullOrWhiteSpace(txtMiddleName->Text))
{
    Regex^ middleNameRegex = gcnew Regex(L"^[А-ЯЁ][а-яё]*(?:-[А-
ЯЁ][а-яё]*)?$");
    if (!middleNameRegex->IsMatch(txtMiddleName->Text))
    {
        MessageBox::Show(L"Отчество должно начинаться с заглавной
буквы и содержать только русские буквы. Допускается дефис");
        txtMiddleName->Focus();
        return false;
    }
}

// Проверка логина
Regex^ loginRegex = gcnew Regex(L"^[a-zA-Z0-9_]{3,50}$");
if (!loginRegex->IsMatch(txtLogin->Text))
{
    MessageBox::Show(L"Логин может содержать только латинские буквы
(A-Z, a-z), цифры (0-9) и символ подчёркивания (_). Длина от 3 до 50
символов.");
    txtLogin->Focus();
    return false;
}

```



```

// Проверка пароля (только при добавлении)
if (!isEditMode)
{
    if (String.IsNullOrEmpty(txtPassword->Text))
    {
        MessageBox::Show(L"Введите пароль!");
        txtPassword->Focus();
        return false;
    }

    if (txtPassword->Text->Length < 4 || txtPassword->Text->Length >
255)
    {
        MessageBox::Show(L"Пароль должен содержать от 4 до 255
СИМВОЛОВ!");
        txtPassword->Focus();
        return false;
    }

    if (txtPassword->Text != txtConfirmPassword->Text)
    {
        MessageBox::Show(L"Пароли не совпадают!");
        txtConfirmPassword->Focus();
        return false;
    }
}

// Проверка выбора должности
if (cmbPosition->SelectedIndex == -1)
{
    MessageBox::Show(L"Выберите должность!");
    cmbPosition->Focus();
    return false;
}

return true;
}

void RegisterForm::OnSaveClick(Object^ sender, EventArgs^ args)
{
    if (!ValidateInputs()) return;

    try
    {
        if (isEditMode)
        {

```

```

SqlCommand^ command = gcnew SqlCommand(L"sp_UpdateEmployee",
sqlConnection);

command->CommandType = CommandType::StoredProcedure;
command->Parameters->AddWithValue(L"@EmployeeID",
editingEmployeeId);
command->Parameters->AddWithValue(L"@LastName", txtLastName-
>Text);

command->Parameters->AddWithValue(L"@FirstName",
txtFirstName->Text);
command->Parameters->AddWithValue(L"@MiddleName",
String::IsNullOrWhiteSpace(txtMiddleName->Text) ? DBNull::Value :
(Object^) txtMiddleName->Text);
command->Parameters->AddWithValue(L"@PositionName",
cmbPosition->SelectedItem->ToString());
command->Parameters->AddWithValue(L"@LoginName", txtLogin-
>Text);

command->Parameters->AddWithValue(L"@UserRole", cmbRole-
>SelectedItem->ToString());
command->Parameters->AddWithValue(L"@IsActive", chkIsActive-
>Checked);

command->ExecuteNonQuery();

MessageBox::Show(gcnew System::String(L"Данные обновлены!"),
gcnew System::String(L"Успех"));
isRegistered = true;
this->Close();
}
else
{
SqlCommand^ command = gcnew SqlCommand(L"sp_AddEmployee",
sqlConnection);

command->CommandType = CommandType::StoredProcedure;
command->Parameters->AddWithValue(L"@LastName", txtLastName-
>Text);

command->Parameters->AddWithValue(L"@FirstName",
txtFirstName->Text);
command->Parameters->AddWithValue(L"@MiddleName",
String::IsNullOrWhiteSpace(txtMiddleName->Text) ? DBNull::Value :
(Object^) txtMiddleName->Text);
command->Parameters->AddWithValue(L"@PositionName",
cmbPosition->SelectedItem->ToString());
command->Parameters->AddWithValue(L"@LoginName", txtLogin-
>Text);

command->Parameters->AddWithValue(L"@Password", txtPassword-
>Text);

command->Parameters->AddWithValue(L"@UserRole", cmbRole-
>SelectedItem->ToString());

```

```

        newEmployeeId = Convert.ToInt32(command->ExecuteScalar());
        MessageBox::Show(gcnew System::String(L"Сотрудник
зарегистрирован! ID: ") + newEmployeeId, gcnew System::String(L"Успех"));
        isRegistered = true;
        this->Close();
    }
}
catch (Exception^ ex)
{
    if (ex->Message->Contains(L"UNIQUE") || ex->Message-
>Contains(L"Duplicate"))
        MessageBox::Show(gcnew System::String(L"Логин уже
существует!"), gcnew System::String(L"Ошибка"));
    else
        MessageBox::Show(gcnew System::String(L"Ошибка: ") + ex-
>Message, gcnew System::String(L"Ошибка"));
}
}

void RegisterForm::OnCancelClick(Object^ sender, EventArgs^ args)
{
    this->Close();
}

```

Приложение Н. Листинг SQL-скрипта TimeTracking.sql

```

CREATE DATABASE TimeTracking;
GO

USE TimeTracking;
GO

--Справочник должностей
CREATE TABLE Position (
    Position_ID INT PRIMARY KEY IDENTITY(1,1),
    Position_Name NVARCHAR(100) NOT NULL UNIQUE
);
GO

--Таблица сотрудников
CREATE TABLE Employee (
    Employee_ID INT PRIMARY KEY IDENTITY(1,1),

    Last_Name NVARCHAR(50) NOT NULL,
    First_Name NVARCHAR(50) NOT NULL,
    Middle_Name NVARCHAR(50) NULL,

    --Организационная структура
    Position_ID INT NOT NULL,

    --Учётные данные для входа
    Login_Name NVARCHAR(50) NOT NULL UNIQUE,
    Password_ NVARCHAR(255) NOT NULL,

    --Роль
    Role_Name NVARCHAR(20) NOT NULL CHECK (Role_Name IN ('Admin',
'Employee')),

    --Метаданные
    Hire_Date DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE), --Дата
приёма на работу (по умолчанию текущая)
    Is_Active BIT NOT NULL DEFAULT 1, --Работает сотрудник или нет

    -- Внешние ключи
    CONSTRAINT FK_Employee_Position FOREIGN KEY (Position_ID) REFERENCES
Position(Position_ID)

);
GO

--Таблица отметок времени

```

```

CREATE TABLE TimeEntry (
    TimeEntry_ID BIGINT PRIMARY KEY IDENTITY(1,1),

    Employee_ID INT NOT NULL,
    Entry_Type NVARCHAR(10) NOT NULL CHECK (Entry_Type IN ('CheckIn',
'CheckOut')),
    Entry_Time DATETIME NOT NULL DEFAULT GETDATE(),

    CONSTRAINT FK_TimeEntry_Employee FOREIGN KEY (Employee_ID) REFERENCES
Employee(Employee_ID),

    -- Индекс для быстрого поиска по сотруднику и дате
    INDEX IX_TimeEntry_Employee_Time (Employee_ID, Entry_Time)
);
GO

```

```

INSERT INTO Employee (Last_Name, First_Name, Position_ID, Login_Name,
Password_, Role_Name, Is_Active)
SELECT N'Admin', N'System', Position_ID, 'admin', 'admin123', 'Admin', 1
FROM Position WHERE Position_Name = N'Администратор приложения';
GO

```

```

-- 1. Добавление сотрудника
CREATE OR ALTER PROCEDURE sp_AddEmployee
    @LastName NVARCHAR(50),
    @FirstName NVARCHAR(50),
    @MiddleName NVARCHAR(50) = NULL,
    @PositionName NVARCHAR(100),
    @LoginName NVARCHAR(50),
    @Password NVARCHAR(255),
    @UserRole NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @PositionID INT;

    SELECT @PositionID = Position_ID FROM Position WHERE Position_Name =
@PositionName;

    IF @PositionID IS NULL
    BEGIN
        INSERT INTO Position (Position_Name) VALUES (@PositionName);
        SET @PositionID = SCOPE_IDENTITY();
    END
END

```

```

        INSERT INTO Employee (Last_Name, First_Name, Middle_Name,
        Position_ID, Login_Name, Password_, Role_Name, Is_Active)
        VALUES (@LastName, @FirstName, @MiddleName, @PositionID, @LoginName,
        @Password, @UserRole, 1);

```

```

        SELECT SCOPE_IDENTITY() AS NewEmployeeID;
    END
    GO

```

-- 2. Обновление сотрудника

```

CREATE OR ALTER PROCEDURE sp_UpdateEmployee
    @EmployeeID INT,
    @LastName NVARCHAR(50),
    @FirstName NVARCHAR(50),
    @MiddleName NVARCHAR(50) = NULL,
    @PositionName NVARCHAR(100),
    @LoginName NVARCHAR(50),
    @UserRole NVARCHAR(20),
    @IsActive BIT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @PositionID INT;

    SELECT @PositionID = Position_ID FROM Position WHERE Position_Name =
    @PositionName;

```

```

    IF @PositionID IS NULL
    BEGIN
        INSERT INTO Position (Position_Name) VALUES (@PositionName);
        SET @PositionID = SCOPE_IDENTITY();
    END

```

```

    UPDATE Employee
    SET Last_Name = @LastName,
        First_Name = @FirstName,
        Middle_Name = @MiddleName,
        Position_ID = @PositionID,
        Login_Name = @LoginName,
        Role_Name = @UserRole,
        Is_Active = @IsActive
    WHERE Employee_ID = @EmployeeID;
END
GO

```

-- 3. Удаление сотрудника (каскадное)

```

CREATE OR ALTER PROCEDURE sp_DeleteEmployee

```

```

        @EmployeeID INT
    AS
    BEGIN
        SET NOCOUNT ON;
        DELETE FROM TimeEntry WHERE Employee_ID = @EmployeeID;
        DELETE FROM Employee WHERE Employee_ID = @EmployeeID;
    END
GO

-- 4. Добавление отметки времени
CREATE OR ALTER PROCEDURE sp_AddTimeEntry
    @EmployeeID INT,
    @EntryType NVARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO TimeEntry (Employee_ID, Entry_Type, Entry_Time)
    VALUES (@EmployeeID, @EntryType, GETDATE());
    SELECT SCOPE_IDENTITY() AS NewEntryID;
END
GO

-- 5. Аутентификация пользователя
CREATE OR ALTER PROCEDURE sp_AuthenticateUser
    @LoginName NVARCHAR(50),
    @Password NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Employee_ID, First_Name, Last_Name, Role_Name
    FROM Employee
    WHERE Login_Name = @LoginName
        AND Password_ = @Password
        AND Is_Active = 1;
END
GO

-- 6. Получение отчёта за период
CREATE OR ALTER PROCEDURE sp_GetTimeReport
    @StartDate DATE,
    @EndDate DATE
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        e.Employee_ID,

```

```

        e.Last_Name + ' ' + e.First_Name + ISNULL(' ' + e.Middle_Name,
        '') AS FullName,
        p.Position_Name AS Position,
        CAST(te.Entry_Time AS DATE) AS WorkDate,
        MAX(CASE WHEN te.Entry_Type = 'CheckIn' THEN te.Entry_Time END)
AS CheckInTime,
        MAX(CASE WHEN te.Entry_Type = 'CheckOut' THEN te.Entry_Time END)
AS CheckOutTime
FROM Employee e
INNER JOIN Position p ON e.Position_ID = p.Position_ID
INNER JOIN TimeEntry te ON e.Employee_ID = te.Employee_ID
WHERE CAST(te.Entry_Time AS DATE) BETWEEN @StartDate AND @EndDate
AND e.Is_Active = 1
GROUP BY e.Employee_ID, e.Last_Name, e.First_Name, e.Middle_Name,
p.Position_Name, CAST(te.Entry_Time AS DATE)
ORDER BY e.Last_Name, e.First_Name, WorkDate;
END
GO

```

-- 7. Получение всех сотрудников

```

CREATE OR ALTER PROCEDURE sp_GetAllEmployees
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        e.Employee_ID,
        e.Last_Name,
        e.First_Name,
        e.Middle_Name,
        p.Position_Name AS Position,
        e.Login_Name,
        e.Role_Name,
        e.Hire_Date,
        e.Is_Active
    FROM Employee e
    INNER JOIN Position p ON e.Position_ID = p.Position_ID
    ORDER BY e.Last_Name, e.First_Name;
END
GO

```

-- 8. Получение отметок сотрудника за период

```

CREATE OR ALTER PROCEDURE sp_GetTimeEntriesForPeriod
    @EmployeeID INT,
    @StartDate DATE,
    @EndDate DATE
AS
BEGIN

```



```

SET NOCOUNT ON;
SELECT
    TimeEntry_ID,
    Entry_Type,
    Entry_Time,
    FORMAT(Entry_Time, 'dd.MM.yyyy') AS EntryDate,
    FORMAT(Entry_Time, 'HH:mm:ss') AS EntryTimeOnly
FROM TimeEntry
WHERE Employee_ID = @EmployeeID
    AND CAST(Entry_Time AS DATE) BETWEEN @StartDate AND @EndDate
ORDER BY Entry_Time DESC;
END
GO

```

-- 9. Получение последней отметки сотрудника

```

CREATE OR ALTER PROCEDURE sp_GetLastTimeEntry
    @EmployeeID INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT TOP 1
        Entry_Type,
        Entry_Time,
        FORMAT(Entry_Time, 'dd.MM.yyyy HH:mm:ss') AS FormattedTime
    FROM TimeEntry
    WHERE Employee_ID = @EmployeeID
    ORDER BY Entry_Time DESC;
END
GO

```

-- 10. Получение всех должностей

```

CREATE OR ALTER PROCEDURE sp_GetAllPositions
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Position_ID, Position_Name
    FROM Position
    ORDER BY Position_Name;
END
GO

```

```

INSERT INTO Position (Position_Name) VALUES
(N'Председатель'),
(N'Заместитель председателя'),
(N'Помощник председателя'),
(N'Начальник отдела'),

```

```

(N'Заместитель начальник отдела'),
(N'Судья'),
(N'Помощник судьи'),
(N'Администратор приложения'),
(N'Секретарь зала заседаний'),
(N'Администратор суда');
GO

INSERT INTO Employee (Last_Name, First_Name, Position_ID, Login_Name,
Password_, Role_Name, Is_Active)
SELECT N'Admin', N'System', Position_ID, 'admin', 'admin123', 'Admin', 1
FROM Position WHERE Position_Name = N'Администратор приложения';
GO

-- Включение смешанной аутентификации
EXEC xp_instance_regwrite N'HKEY_LOCAL_MACHINE',
N'Software\Microsoft\MSSQLServer\MSSQLServer', N'LoginMode', REG_DWORD, 2;
GO

-- Включение учётной записи sa и установка пароля
ALTER LOGIN sa ENABLE;
GO
ALTER LOGIN sa WITH PASSWORD = N'123456', CHECK_POLICY = OFF;
GO

EXEC xp_create_subdir 'C:\employee_time_tracking\Backup\TimeTracking.bak'

BACKUP DATABASE TimeTracking TO DISK =
'C:\employee_time_tracking\Backup\TimeTracking.bak'

```

Приложение О. Листинг файла CMakeLists.txt

```

cmake_minimum_required(VERSION 3.20)
project(TimeTracking VERSION 1.0 LANGUAGES CXX)

if(NOT MSVC)
    message(FATAL_ERROR "C++/CLI projects require MSVC compiler")
endif()

set(SOURCE_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../src")
set(INCLUDE_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../include")

include_directories(${INCLUDE_DIR})

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY_DEBUG "${CMAKE_BINARY_DIR}/bin/Debug")
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY_RELEASE
"${CMAKE_BINARY_DIR}/bin/Release")

file(MAKE_DIRECTORY ${CMAKE_RUNTIME_OUTPUT_DIRECTORY_DEBUG})
file(MAKE_DIRECTORY ${CMAKE_RUNTIME_OUTPUT_DIRECTORY_RELEASE})

# Настройки компилятора (ДОБАВЛЕН ФЛАГ /utf-8)
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /clr /utf-8")
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(TimeTracking
    ${SOURCE_DIR}/LoginForm.cpp
    ${SOURCE_DIR}/Main.cpp
    ${SOURCE_DIR}/MainFormAdmin.cpp
    ${SOURCE_DIR}/RegisterForm.cpp
    ${SOURCE_DIR}/Role.cpp
    ${INCLUDE_DIR}/all_includes.h
    ${INCLUDE_DIR}/LoginForm.h
    ${INCLUDE_DIR}/Main.h
    ${INCLUDE_DIR}/MainFormAdmin.h
    ${INCLUDE_DIR}/RegisterForm.h
    ${INCLUDE_DIR}/Role.h
)

# Настройки для .NET Framework
set_target_properties(TimeTracking PROPERTIES
    VS_DOTNET_TARGET_FRAMEWORK_VERSION "v4.7.2"
    COMMON_LANGUAGE_RUNTIME ""
    WIN32_EXECUTABLE ON
)

# Рабочая директория для отладки

```

```

    set_target_properties(TimeTracking PROPERTIES
        VS_DEBUGGER_WORKING_DIRECTORY
        "${CMAKE_RUNTIME_OUTPUT_DIRECTORY_DEBUG}"
        VS_DEBUGGER_ENVIRONMENT
        "PATH=${CMAKE_RUNTIME_OUTPUT_DIRECTORY_DEBUG}"
        VS_CONFIGURATION_TYPE "Application"
    )
    set_property(TARGET TimeTracking PROPERTY
        VS_DOTNET_REFERENCES
        "System"
        "System.Windows.Forms"
        "System.Drawing"
        "System.Data"
        "System.ComponentModel.Composition"
        "System.ComponentModel.Composition.Registration"
        "System.ComponentModel.DataAnnotations"
        "System.Drawing.Design"
        "System.Management"
    )
    target_link_libraries(TimeTracking
        odbc32
        sqlsrv
    )
    # Дополнительные настройки
    set_property(TARGET TimeTracking PROPERTY VS_WINRT_COMPONENT OFF)
    set_property(TARGET TimeTracking PROPERTY VS_NO_SOLUTION_DEPLOY ON)

    # Установка проекта как стартового
    set_property(DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR} PROPERTY
        VS_STARTUP_PROJECT TimeTracking)

    # Включить папки в обозревателе решений
    set_property(GLOBAL PROPERTY USE_FOLDERS ON)

    # Настройки для Debug/Release
    if(CMAKE_BUILD_TYPE STREQUAL "Debug")
        set_target_properties(TimeTracking PROPERTIES
            COMPILE_DEFINITIONS "DEBUG;_DEBUG"
            MSVC_RUNTIME_LIBRARY "MultiThreadedDebugDLL"
        )
    else()
        set_target_properties(TimeTracking PROPERTIES
            COMPILE_DEFINITIONS "NDEBUG"
            MSVC_RUNTIME_LIBRARY "MultiThreadedDLL"
        )
    endif()

```

